

Industrial Component Anomaly Detection

Benjamin Förster Ali Abul Hawa Mahboubeh Nadaf Karim Khalifa

September 1, 2026

Abstract

This report documents our complete process of developing unsupervised anomaly detection models for industrial components, evaluated on the MVTec Anomaly Detection dataset. We describe the full trajectory of our work, from exploratory data analysis and iterative prototype experiments to a production-grade software architecture, and we explain each decision in the context of the findings that motivated it. Our final system comprises two complementary approaches: an advanced Keras convolutional autoencoder incorporating masked image modelling and a combined structural and pixel-wise loss function, and a PatchCore model with a ResNet-18 backbone. The choice of ResNet-18 was necessitated by the computational and memory constraints of consumer-grade hardware. Both systems are equipped with category-adaptive preprocessing and automated hyperparameter optimisation via Optuna. A later protocol audit found that the historical cross-model evaluation did not consistently enforce the intended 85% training/15% validation split: the CAE used the split, whereas PatchCore built its memory bank from the complete normal training partition. We therefore treat the affected comparison as provisional and define a corrected zero-leakage protocol in which both models use the same split and the test set is accessed only for final evaluation. The primary metrics are the F1 score at image level and the Area Under the Per-Image Overlap (AUPIMO) at pixel level, complemented by precision-recall curves and threshold sensitivity curves.

1 Introduction and Task Description

This project was conducted as part of the Liora data science curriculum and with a difficulty rating of 10 out of 10. The objective was to develop a model capable of detecting anomalies in industrial components from images, using the MVTec AD dataset [1, 2]. The task description listed two expected deliverables: a report on data exploration, visualisation, and preprocessing, and a final report together with an associated code repository. This document serves as the final report, covering the complete project from data exploration to the evaluation of the finished system. The task description explicitly references the paper *AUPIMO: Redefining Visual Anomaly Detection Benchmarks with High Speed and Low Tolerance* [5] as a key bibliographic source. This motivated our adoption of AUPIMO as the primary pixel-level evaluation metric, a choice we justify technically in Section 3.

Quality control in modern manufacturing operates under a fundamental constraint: defects are rare by design. A well-engineered production line yields thousands of good items for every defect. This scarcity makes supervised classification impractical in the classical sense, as there are simply not enough labelled defect examples to train a reliable closed-world classifier, and future defects take forms that were never seen during development. The standard response to this challenge is unsupervised anomaly detection. Our models train on defect-free normal images and learn compact representations of what normal looks like. At test time, any image that deviates significantly from this learned normal appearance is flagged as anomalous. This is both technically demanding and fundamentally different from standard classification, because

the model must determine what is normal and anomalous from normal-only training data, with no exposure to any defect during training.

Throughout this report, we describe our work in the order in which we executed it, including our iterative challenges and course corrections. We believe this honest account of the learning process is as informative as the final results.

2 Exploratory Data Analysis

Before writing a single line of model code, we invested substantial effort in understanding the dataset. The full Exploratory Data Analysis report with all corresponding visualisations was submitted separately. The insights from this phase directly shaped every subsequent technical decision. Many properties of the data that we uncovered would have silently degraded our models.

2.1 Dataset Composition and Integrity

We began by building a complete dataset manifest. This is a structured table listing every image and its corresponding metadata, including product category, data split, defect type, pixel dimensions, and the path to its ground-truth mask. This manifest served as the foundation for all subsequent analyses and allowed us to run systematic integrity checks before touching any model code.

We verified that every defective test image is paired with exactly one ground-truth mask, that all image-mask dimension pairs match, and that all masks are stored in greyscale format with binary-valued pixels. We used SHA-256 hashing to check for byte-for-byte duplicates and confirmed that no exact duplicate images exist and that there is no image-level train-to-test leakage in the dataset.

In total, the dataset contains 3,629 normal training images, 467 normal test images, and 1,258 defective test images spread across 15 product categories. The training split consists exclusively of defect-free images, reflecting the unsupervised detection paradigm. We found that image modes are heterogeneous across categories: most categories contain RGB images, while the categories *grid*, *screw*, *tile*, *toothbrush*, and *zipper* use greyscale images. Image resolutions range from 700×700 to 1024×1024 pixels, and all images are square. This heterogeneity in colour mode and spatial resolution made explicit format handling and a standardised loading pipeline a necessity, as we describe in Section 4.

2.2 Objects versus Textures: A Fundamental Structural Split

The 15 categories divide into two structurally distinct groups that require entirely different treatment throughout the pipeline. The five texture categories (Carpet, Grid, Leather, Tile, and Wood) consist of repetitive, spatially invariant surface patterns. A patch of leather looks statistically the same regardless of whether it has been rotated or mirrored. The ten object categories (bottle, cable, capsule, hazelnut, metal_nut, pill, screw, toothbrush, transistor, and zipper) are physically shaped items that arrive at the inspection stage in a consistent orientation. Applying aggressive rotational augmentation to a bottle would incorrectly instruct the model that inverted orientations are part of the normal distribution, rendering the detector blind to orientation-based anomalies.

We identified this distinction on the first day of the project and carried its implications through to augmentation, foreground extraction, and configuration tuning.

2.3 Pixel-Level Class Imbalance and the Accuracy Paradox

We computed pixel-level defect statistics across all 1,258 defective test images. The median defect occupies only 1.54% of the image area. The 25th percentile is 0.49%, meaning that one quarter of all defects cover less than half a percent of the image. Some defects are as small as 0.037% of the image area, which amounts to a barely visible scratch or pinhole. The mean of 4.39% is inflated by a small number of large structural defects that reach up to 49.32% of the image.

These numbers carry a direct consequence for evaluation. A model that classifies every pixel in every image as normal achieves over 98% pixel-level accuracy on a typical defective image, while detecting exactly zero defects. Standard accuracy is therefore a completely misleading metric for this problem, which is what the literature refers to as the accuracy paradox. We concluded from this analysis that we needed to use metrics that are sensitive to the minority positive class and that do not get inflated by the large pool of correctly-ignored normal pixels. Precision-recall curves and the F1 score have exactly this property, since they are computed entirely from true positives, false positives, and false negatives. The pool of true negatives (normal pixels) never enters their calculation.

2.4 Statistical Proof of Category Heterogeneity

A recurring question during the EDA was whether a single binarisation threshold could be applied uniformly across all 15 categories. We answered this question with two statistical tests.

We first ran the Kruskal-Wallis H-Test, which tests the null hypothesis that the population medians of image-level defect area ratios are equal across all 15 categories. We chose the Kruskal-Wallis test over a pixel-level Chi-Square test for methodological reasons: pixels are strongly spatially correlated, which violates the independence assumption required by Chi-Square. Furthermore, the massive pixel sample size ($N > 67$ million) would cause the Chi-Square test to flag even negligibly small differences as statistically significant, an example of inflated statistical power overwhelming practical significance. The Kruskal-Wallis test treats each defective test image as a single independent observation ($N = 1,258$), making the inference trustworthy. The result was $H = 626.5$ with $p = 1.2 \times 10^{-124}$, a decisive rejection of the null hypothesis. Defect size distributions differ significantly across categories.

As a complementary test, we ran a pixel-level Chi-Square test to assess whether the overall proportion of anomalous pixels differs across categories. The test yielded $\chi^2(14) \approx 41.7$ million with $p \approx 0$. The anomalous-pixel ratios range from 0.34% for *screw* to 14.5% for *metal_nut*, which is a 43-fold spread.

The combined result is unambiguous: a single detection threshold applied uniformly across all 15 categories is statistically indefensible. This finding motivated our decision to tune preprocessing and model hyperparameters separately per category using Optuna.

2.5 Defect Shape, Morphology, and Spatial Location

We extended the defect analysis beyond area to include shape complexity and spatial location. For each ground-truth mask, we computed the bounding-box fill ratio, which is the fraction of the bounding box occupied by the defect, and the number of connected components, which indicates whether the defect is a single region or multiple disconnected fragments.

We found that defects vary widely in all of these dimensions. Some defects are extremely thin relative to their bounding box, for example hairline cracks in the *capsule* category. Others

consist of multiple disconnected fragments, as is common in the *cable* category where damage occurs at several points simultaneously. Several categories contain defects that touch the image border, which means that any preprocessing step that crops or pads the image boundary could destroy part of the ground-truth annotation.

We also mapped the normalised spatial centre-of-mass coordinates of each defect. Rigid-object categories, particularly *bottle* and *screw*, show strong positional clustering of defects, reflecting the physical reality that certain component surfaces are more susceptible to certain types of damage. Texture categories, by contrast, show broadly distributed defect locations. We noted this positional bias as a potential shortcut for a model to exploit. A model that learns to flag the top of a bottle rather than flagging structurally anomalous regions would score well on the benchmark but fail completely in real deployment. Evaluation and augmentation decisions must be designed to prevent this.

2.6 Normal Train/Test Covariate Shift

We compared brightness and contrast distributions between normal training images and normal test images within each category. Since both sets are defect-free, any systematic difference in these statistics represents a domain shift caused by acquisition conditions rather than genuine anomalies. A model that learns the specific lighting conditions of the training set may flag normal test images as anomalous simply because the illumination differs.

We found noticeable differences in brightness or contrast for the *screw*, *leather*, and *grid* categories. This finding had two consequences: it reinforced the need for per-category evaluation rather than global averaging, and it motivated the inclusion of photometric augmentation (brightness and contrast variation) during training to make the model robust to acquisition-level variation.

2.7 Why We Do Not Use Pixel-Level AUROC

The standard metric for anomaly map quality in the literature is the pixel-level AUROC. We examined this metric carefully during the EDA and concluded that it is unsuitable as a primary evaluation standard for MVTec AD, for the same reason that accuracy is unsuitable. AUROC is built on the false positive rate, defined as $FPR = FP / (FP + TN)$. Since true negatives (normal pixels) constitute approximately 98.5% of all pixels in a typical defective test image, a model can misclassify substantial numbers of pixels while the FPR barely moves. This keeps AUROC deceptively high even for weak detectors.

Published state-of-the-art results confirm this: PatchCore reports pixel AUROC of 99.6%, EfficientAD up to 99.8%, and Dinomaly 99.6%. When top systems differ by only a few tenths of a percent, AUROC cannot distinguish model quality. We therefore use pixel-level AUROC only as an internal proxy during Optuna optimisation trials where computational speed matters, and we report AUPIMO as the primary localisation metric in all final evaluations.

2.8 Threshold Sensitivity and the Business Case

As a final EDA step, we analysed the breakpoint phenomenon in threshold selection. Every anomaly detection model outputs a continuous anomaly score per pixel, and deployment requires a binary decision threshold t that separates normal pixels from anomalous ones. Selecting this threshold is not a purely statistical decision, it is a business decision with direct financial consequences.

A threshold set too high produces high precision, meaning there are few false alarms and the production line is not stopped unnecessarily, but low recall, meaning real defects escape undetected and reach customers. A threshold set too low catches more defects and yields high recall, but triggers frequent unnecessary line stoppages, which leads to high cost from lost throughput. We computed the F1 score, which balances precision and recall, the F2 score, which places twice the weight on recall for applications where missed defects are unacceptably costly, and the F0.5 score, which places twice the weight on precision for applications where false alarms dominate costs. This allowed us to map the full trade-off quantitatively. The threshold sensitivity analysis showed that precision and recall react sharply to small threshold changes, making the choice of operating point non-trivial even within a single category.

3 Evaluation Metrics

Based on the findings of the EDA, we selected a set of evaluation metrics that operate at two distinct levels, image-level and pixel-level, and that are honest under the extreme class imbalance of the dataset.

3.1 Image-Level: F1 Score and Precision-Recall Curve

At image level, the task is binary classification. Given a test image, we want to know whether it is normal or defective. We compute the F1 score, defined as the harmonic mean of Precision P and Recall R :

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}, \quad \text{where} \quad P = \frac{TP}{TP + FP} \quad \text{and} \quad R = \frac{TP}{TP + FN}.$$

Here, a true positive (TP) is a defective image that the model correctly flags, a false positive (FP) is a normal image that the model incorrectly flags, and a false negative (FN) is a defective image that the model misses. The denominator of precision contains only images that the model actually flagged. It does not involve the large pool of correctly-accepted normal images. This makes F1 immune to the accuracy paradox. The image-level classification threshold is derived exclusively from normal validation images, as described in Section 7.3.

We complement the F1 score with the full precision-recall curve, which traces the trade-off across all possible threshold values. The area under this curve summarises ranking quality in a single number that is fully sensitive to the imbalance between normal and defective images.

3.2 Pixel-Level: AUPIMO

At pixel level, the task is to produce an anomaly map $a \in \mathbb{R}^{H \times W}$ such that anomalous pixels receive high scores and normal pixels receive low scores. We use AUPIMO [5] as the primary localisation metric.

AUPIMO is built on two core ideas. The first is normal-only threshold calibration. For a given threshold t , the shared false positive rate $F_{sh}(t)$ is computed exclusively over the set of strictly normal images $\mathcal{Y}^0$:

$$F_{sh}(t) = \frac{1}{|\mathcal{Y}^0|} \sum_{y \in \mathcal{Y}^0} \frac{|\{(i, j) : a_{ij}^{(y)} \geq t\}|}{H \cdot W}.$$

The threshold calibration is completely blind to anomalous test images. It cannot access ground-truth defect locations, and there is no risk of the threshold selection leaking information about where defects are.

The second idea is per-image scoring. Rather than pooling all pixels from all test images into a single global FPR-TPR trade-off, AUPIMO computes the true positive rate $T_i(t)$ independently for each anomalous image $y_i \in \mathcal{Y}^1$:

$$T_i(t) = \frac{|\{(j, k) : a_{jk}^{(y_i)} \geq t\} \cap \{(j, k) : m_{jk}^{(y_i)} = 1\}|}{|\{(j, k) : m_{jk}^{(y_i)} = 1\}|},$$

where $m^{(y_i)}$ is the binary ground-truth mask of image y_i . Every image contributes equally to the final score regardless of its defect size.

The final AUPIMO score is computed as the integral of T_i over a logarithmically-spaced range of Shared FPR values:

$$\text{AUPIMO}_i = \frac{1}{\ln(U/L)} \int_L^U T_i(F_{sh}^{-1}(z)) \frac{dz}{z}, \quad L = 10^{-5}, U = 10^{-4}.$$

The integration bounds $L = 10^{-5}$ and $U = 10^{-4}$ are chosen to reflect the operating regime of real industrial deployment. For a 256×256 image containing 65,536 pixels, $F_{sh} = 10^{-5}$ corresponds to fewer than one false positive pixel per image on average. Evaluating in this regime is what distinguishes AUPIMO from AUROC, because state-of-the-art models that score 99% under pixel AUROC typically score 60-70% under AUPIMO, revealing that current benchmarks are far from solved at industrially meaningful false positive rates.

3.3 Threshold Sensitivity Curves

For each model, we also report threshold sensitivity curves that show how F1 and AUPIMO evolve as the classification threshold t varies. These curves reveal whether a model’s performance is stable across a range of thresholds or collapses abruptly at a specific threshold value. A robust threshold curve is preferable in deployment, because real production conditions always involve some drift in image characteristics over time.

4 Preprocessing Pipeline

All images pass through a standardised preprocessing pipeline before entering any model. We designed this pipeline in direct response to the EDA findings concerning the heterogeneity of image formats, the object-versus-texture distinction, the risk of covariate shift from background variation, and the fragility of small defects to aggressive resizing.

4.1 Loading, Format Standardisation, and Resizing

We load every image using PIL and immediately convert it to three-channel RGB. This step is mandatory because MVTec images exist in four distinct formats: RGB JPEG, RGBA PNG, 8-bit greyscale PNG, and palette-mapped PNG. Without explicit conversion, feeding a 1-channel greyscale image to a network expecting 3 channels would produce a shape mismatch.

We resize all images to 256×256 pixels using LANCZOS resampling. LANCZOS implements a windowed sinc function (mathematically defined as $\text{sinc}(x) = \frac{\sin(x)}{x}$), which acts as an ideal mathematical low-pass filter. This mathematically minimises aliasing artefacts while preserving edge sharpness and high-frequency texture detail. We chose it specifically over bilinear or bicubic resampling because some defects are microscopic, and a lower-quality interpolation filter could destroy that signal entirely during downscaling.

Ground-truth defect masks are resized separately using nearest-neighbour interpolation. We deliberately do not use anti-aliasing filters for masks because anti-aliasing would blur the binary mask boundaries, introducing fractional pixel values at the edges of annotated defect regions. After resizing, we binarise all masks at the threshold 127 to ensure strict binary arrays.

4.2 Category-Aware Augmentation

We apply data augmentation to the training set only, in order to increase the effective diversity of the training distribution. The augmentation strategy differs fundamentally between texture and object categories, as motivated by the EDA.

For texture categories, we apply heavy spatial augmentation. We randomly rotate images in 90 degree increments, flip them horizontally and vertically, and apply scale jitter by cropping a region sized between 80% and 100% of the original image and resizing it back to the target resolution. We additionally vary brightness and contrast by up to 20%. Every spatially augmented version of a normal texture image is a genuinely valid training example, because texture surfaces are statistically indistinguishable at arbitrary orientations.

For object categories, we apply photometric augmentation only. We vary brightness, contrast, and saturation each by up to 10% and add Gaussian noise with standard deviation $\sigma = 0.02$. We do not apply any spatial transformation, because an object’s orientation on the inspection conveyor is part of its normal appearance, and training on arbitrarily oriented objects would destroy the model’s ability to detect orientation-based anomalies.

4.3 Foreground Extraction

Industrial inspection images always include a background such as a conveyor belt surface or a mounting jig. This background wastes model capacity and creates a source of false positives. Any slight variation in the background, like a shadow or a piece of dust, generates a reconstruction error that the model may misidentify as a product defect.

We remove backgrounds using a two-stage classical computer vision pipeline. In the first stage, we apply Otsu’s global thresholding method [10] to the greyscale version of the image. Otsu’s method finds the threshold T^* that maximises the between-class variance:

$$\sigma_B^2(T) = w_0(T) \cdot w_1(T) \cdot [\mu_0(T) - \mu_1(T)]^2,$$

where $w_0(T)$ and $w_1(T)$ are the fractions of pixels assigned to the background class and foreground class respectively, and $\mu_0(T)$ and $\mu_1(T)$ are their corresponding mean intensities. This method works automatically because it adapts to the pixel intensity histogram of each individual image.

In the second stage, we apply Canny edge detection [11] to find the sharp object boundaries that Otsu’s region-growing approach may miss. Canny operates using Gaussian smoothing, Sobel gradient computation, non-maximum suppression, and hysteresis thresholding. We set the two

Canny thresholds adaptively at $\pm 33\%$ around the image’s median pixel intensity, making the detector self-calibrating across varied lighting conditions.

The binary masks produced by Otsu and Canny are merged with a logical OR. We then apply a 5×5 morphological closing operation to fill any remaining holes. Finally, we retain only the largest connected component and discard all others, removing dust particles and small image artefacts. The background is replaced with pure black, ensuring that the model produces zero reconstruction error in background regions.

4.4 Optional Enhancement Steps

The pipeline exposes two additional preprocessing steps that are not applied globally but are toggled per category by the Optuna hyperparameter sweep (Section 8).

The Contrast Limited Adaptive Histogram Equalization (CLAHE) divides the image into a regular grid of 8×8 pixel tiles and applies local histogram equalisation within each tile, with a clip limit of 2.0 that prevents over-amplification of noise in uniform regions. CLAHE enhances subtle surface textures, faint stains, and low-contrast scratches. The associated risk is that CLAHE also amplifies normal microscopic surface variations, which may cause a sensitive model to flag them as structural anomalies.

The Gaussian Blur step applies a 5×5 Gaussian kernel to the image before it enters the model. The blur acts as a low-pass filter that suppresses high-frequency sensor noise and microscopic surface irregularities. The risk is that blurring destroys fine-grained detail, and if the defects being targeted are microscopic, the blur may erase them from the image before the model can detect them.

We observed during initial experiments that applying both CLAHE and Gaussian Blur together can introduce visual artifacts. CLAHE amplifies high-frequency detail, and Gaussian Blur then smears it into a cloud of intermediate-frequency patterns that neither the normal nor the defective training distribution resembles. We therefore let the Optuna sweep determine the combination of these steps that maximises pixel AUROC for each category individually.

5 Exploratory Autoencoder Development

Our first model development phase consisted of iterative notebook-based experiments. We focused exclusively on the *bottle* category to maintain a consistent comparison base. Under the intended deterministic 85%/15% protocol, its 209 normal development images are divided into 177 training images and 32 validation images; the official test set contains 83 images and remains separate. We deliberately chose to document this process as a progression of insights, because the dead ends and missteps are as informative as the improvements.

We developed several autoencoder variants over the course of the exploration phase. We present four representative milestone architectures that capture the key turning points in our understanding, followed by the final production model as a fifth architecture. We made the decision not to discuss every single iteration to avoid obscuring the main conceptual arc of the project.

5.1 The Reconstruction Principle

All autoencoder experiments share the same detection principle. An autoencoder consists of an encoder that compresses the input image into a compact latent representation, and a decoder

that reconstructs the image from the latent code. During training, both networks are jointly optimised to minimise the reconstruction loss on normal images only.

At inference time, the pixel-level reconstruction error serves as the anomaly map. A well-trained autoencoder produces low error on normal images and high error at defective regions, because it has never learned how anomalous structures look and reconstructs the expected normal surface instead.

The image-level anomaly score aggregates the per-pixel error into a single scalar value. To compute the classification threshold, we collect these scalar anomaly scores for all normal validation images, sort them in ascending order, and select the value below which 95% of the scores fall. This 95th percentile then serves as our classification threshold.

5.2 Milestone 1: Three-Stage MSE Autoencoder

Our first architecture, Milestone 1, used a symmetric three-stage convolutional encoder-decoder. The encoder applied three stride-2 convolutions with a kernel size of 4×4 , progressively doubling the number of feature channels from 32 to 128 while halving the spatial dimensions from 256×256 to 32×32 . The decoder mirrored this structure with three transposed-convolution stages. The bottleneck held 131,072 values, giving a compression ratio of approximately 1.5 relative to the input. We used ReLU and LeakyReLU activations during this early phase, which we would later replace with ELU.

Input $256 \times 256 \times 3$	→	Enc-1 Conv(32)	→	Enc-2 Conv(64)	→	Bottleneck $128 \times 32 \times 32$
Bottleneck $128 \times 32 \times 32$	→	Dec-2 ConvT(64)	→	Dec-1 ConvT(32)	→	Output $256 \times 256 \times 3$

We trained with mean squared error (MSE) loss and the Adam optimiser. We extended the training progressively because validation loss was still decreasing at the end of each run, until we achieved the lowest validation reconstruction MSE of the group at 200 epochs.

On the bottle test set, this model produced an image-level AUROC of 0.617 and detected 24 of 63 defects. These are weak results. More strikingly, the version with the best reconstruction quality on normal images was the worst anomaly detector of the entire exploration phase.

From these initial experiments, we discovered the core failure mode of autoencoders trained purely on mean squared error. A low reconstruction error on normal images does not imply a correspondingly high reconstruction error on defective images. A model trained to minimise the mean squared error develops a high capacity to reconstruct any smooth image, including images with defects. Consequently, the reconstruction error becomes uninformative as an anomaly signal. The correct goal is not to achieve the lowest possible reconstruction error on normal images, but rather to maximise the difference in reconstruction error between normal and defective images.

5.3 Milestone 2: First Masked Restoration Experiment

The identity-mapping problem, where the autoencoder learns to copy its input rather than learning the structure of normal images, suggested that the training task itself needed to change. We introduced our first masking strategy in Milestone 2, keeping the same three-stage architecture but replacing the clean reconstruction objective with a masked restoration task. During training, we randomly selected two non-overlapping 32×32 patches in the input image and replaced

them with the mean colour of the image. The model could no longer copy its input, as it had to use the surrounding context to infer what should appear in the masked areas.

We evaluated the trained checkpoint using a clean-input scoring protocol where we fed the original unmasked image through the encoder, and a masked-restoration scoring protocol where we tiled the image with a fixed set of deterministic masks.

Scoring method	AUROC	PR-AUC	Bal. Acc.	Defects found
M1 (clean-input)	0.617	0.860	0.641	24/63
M2 (clean-input)	0.763	0.924	0.766	43/63
M2 (masked-restoration)	0.684	0.900	0.713	30/63

The masked training objective improved clean-input scoring substantially, but the masked-restoration scoring protocol did not outperform clean-input scoring and missed more defects.

This milestone demonstrated that masked training changes what the model learns, but it does not automatically produce better anomaly scores. The scoring protocol is a separate design choice from the training task. An effective training objective paired with a poorly matched scoring method still yields weak results.

5.4 Milestone 3: Stronger Spatial Compression

Milestone 3 introduced a fourth encoder stage that reduces the latent spatial resolution from 32×32 to 16×16 . The bottleneck became $128 \times 16 \times 16 = 32,768$ values, which is a compression factor of 6 relative to the input. The encoder had four stride-2 convolutional stages, and the decoder mirrored it with four transposed-convolution stages.

In $256^2 \times 3$	→	E-1 Conv(32)	→	E-2 Conv(64)	→	E-3 Conv(128)	→	Bottleneck $128 \times 16 \times 16$
Bottleneck $128 \times 16 \times 16$	→	D-3 ConvT(128)	→	D-2 ConvT(64)	→	D-1 ConvT(32)	→	Out $256^2 \times 3$

We kept the MSE loss, clean-input scoring, and the same 200-epoch training schedule. The result was an AUROC of 0.877 and 49 of 63 defects detected. This was the best autoencoder result of the exploration phase by a wide margin.

These results confirmed that stronger compression is the correct architectural lever for reconstruction-based anomaly detection. By squeezing the representation into a 16×16 spatial grid, we forced the model to discard fine-grained local texture detail and retain only the coarser structural grammar of normal bottles. Anomalous structures cannot be faithfully reconstructed from this highly compressed representation, which in turn produces a visible reconstruction error.

We also tested extending the training to 400 epochs, and narrowing the bottleneck further to 32 channels. Both variants performed worse than Milestone 3. Longer training caused the model to over-generalise, and excessive bottleneck narrowing prevented faithful reconstruction even of normal images.

5.5 Milestone 4: Combined Masking and Compression

Milestone 4 combined the masked restoration training objective of Milestone 2 with the deeper four-stage architecture of Milestone 3. We kept the $128 \times 16 \times 16$ bottleneck and the masked-patch loss, and trained for 150 epochs.

Masked-restoration scoring on Milestone 4 achieved an AUROC of 0.822, a clear improvement from combining stronger compression with the masking task. However, it still did not match Milestone 3’s clean-input result of 0.877. The two ideas interacted positively but produced a combined result that fell between their individual bests.

This outcome highlighted that combining two individually beneficial modifications does not guarantee a result that surpasses either one alone. The experiment validated that masking helps more when combined with stronger compression, but the masking strategy still required further refinement before it could outperform the pure compression approach. This insight directly motivated the design of our final Keras autoencoder.

5.6 The Pretrained Baseline: ResNet-18 + 5-NN

During the exploratory phase, it became evident that training an autoencoder from scratch on a highly constrained normal-only dataset is challenging. To properly contextualise the autoencoder results, we established a simple comparative baseline using transfer learning. We extracted global features from a frozen ResNet-18 network pretrained on ImageNet and applied a basic 5-Nearest Neighbour (5-NN) scoring algorithm.

This simple pretrained baseline achieved an AUROC of 0.9881, a PR-AUC of 0.9960, an F1 score of 0.9594, and a balanced accuracy of 0.9433 on the *bottle* test set. At its normal-validation threshold, it detected 59 of 63 defects while raising only one false alarm among 20 normal test images.

This result starkly contrasted with our best autoencoder configuration at the time (AUROC 0.877, 49/63 defects). It provided the decisive proof that frozen ImageNet features dramatically outperform models trained from scratch for this specific task, directly motivating our subsequent adoption of the full PatchCore architecture (Section 7).

6 The Final Keras CAE (Milestone 5)

Based on the four milestones, we designed the final production autoencoder in Keras. This model incorporates the main successful ideas while addressing the remaining weaknesses identified during the exploration phase. We structured the Keras code using a sequential model setup to ensure clean layer definitions and manageable state.

6.1 Architecture and Activation Layers

The encoder uses four stride-2 convolutional stages with 4×4 kernels and a channel progression of $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$. The decoder mirrors the encoder with four transposed-convolution stages of matching channel widths. A final 1×1 convolutional layer reduces the bottleneck from 256 to 32 channels while preserving the 8×8 spatial grid. This fully convolutional bottleneck contains $32 \times 8 \times 8 = 2,048$ values, operating on 128×128 input images to give a compression ratio of approximately 24.

In $128^2 \times 3$	→	E-1 Conv(32) BN+ELU	→	E-2 Conv(64) BN+ELU	→	E-3 Conv(128) BN+ELU	→	E-4 Conv(256) BN+ELU	→	Bottleneck $8 \times 8 \times 32$ FCAE
Bottleneck $8 \times 8 \times 32$ FCAE	→	D-4 ConvT(256) BN+ELU	→	D-3 ConvT(128) BN+ELU	→	D-2 ConvT(64) BN+ELU	→	D-1 ConvT(32) Sigmoid	→	Out $128^2 \times 3$

By calculating the spatial dimensions and channel depths, we can trace the total number of activations (or “neurons”) passed through each stage of the network. The 128×128 RGB input contains 49,152 values. As the spatial dimensions halve and channels double with each stride-2 convolution, the encoder activations progress from 131,072 (E-1) $\rightarrow$ 65,536 (E-2) $\rightarrow$ 32,768 (E-3) $\rightarrow$ 16,384 (E-4), finally condensing into just 2,048 neurons in the $8 \times 8 \times 32$ bottleneck. The decoder then expands this compressed representation, peaking at 524,288 neurons in D-1 before collapsing back to the 49,152 output pixels.

We moved away from the early ReLU and LeakyReLU activations and used Exponential Linear Unit (ELU) activations throughout the network. The ReLU sets its output to zero for all negative inputs, which can cause neurons that consistently receive negative activations to produce zero gradient permanently. ELU avoids this by saturating smoothly toward a negative value for negative inputs, ensuring that the gradient always flows. We use Batch Normalisation after every convolution, which re-centres and re-scales activations within each mini-batch. All convolutional layers omit the bias term, because the Batch Normalisation layer’s learnable shift parameter subsumes any constant bias.

The bottleneck is fully convolutional. Rather than flattening the feature maps into a 1D vector, we apply an additional 1×1 convolution that reduces the channel count while retaining the 8×8 spatial grid. During our manual testing on the bottle dataset, we actually tried using a central flatten layer followed by a dense layer for the bottleneck. We observed that this produced a good binary classification score for whether the image as a whole was anomalous or not, but it completely scrambled the spatial localisation map. The decoder had to re-learn coordinate relationships from the flattened vector from scratch. By preserving the spatial topology in the fully convolutional bottleneck, we allowed the decoder to maintain and refine spatial relationships during reconstruction.

6.2 Masked Image Modelling and Randomisation

We replaced the coarse two-patch masking of Milestone 2 with a principled Masked Image Modelling strategy, implemented as a custom randomisation layer in the sequential Keras pipeline. The layer divides the image into a regular grid of non-overlapping 16×16 patches. At each training step, the randomisation layer randomly selects 25% of these patches and sets their pixel values to zero. The encoder receives the masked image, and the training target is the original unmasked image.

The random patch selection is re-drawn independently for every image and every training step, ensuring that no patch position is consistently masked. At inference time, the randomisation layer is disabled, and the test image is passed through the network unmodified.

6.3 Combined SSIM and MSE Loss

We replaced the pure MSE training loss with a combination of the structural similarity index measure (SSIM) and MSE. The combined loss is defined as:

$$\mathcal{L}(x, \hat{x}) = \alpha \cdot (1 - \text{SSIM}(x, \hat{x})) + (1 - \alpha) \cdot \text{MSE}(x, \hat{x}), \quad \alpha = 0.84.$$

SSIM evaluates image quality through three complementary local statistics computed over a sliding window. Because SSIM compares local structural patterns rather than individual pixel values, it is robust to the kind of sub-pixel spatial shifts that cause pure pixel-wise losses to be uniformly elevated.

We note that the original publication by Bergmann et al. [3] evaluated SSIM in combination with the L1 loss (Mean Absolute Error) and L2 loss (MSE) and recommended L1 for their specific configuration. We implemented MSE in combination with SSIM because it provided excellent convergence in our Keras pipeline, covering absolute photometric errors while SSIM handles structural differences. The weight parameter $\alpha = 0.84$ controls the balance between structural fidelity and per-pixel accuracy.

6.4 Optimiser and Callbacks

We moved from the standard Adam optimiser to the AdamW optimiser [7]. Standard Adam incorporates weight decay by adding it to the gradient before the gradient enters the adaptive moment estimates. This causes weights with large gradient histories to be under-regularised. AdamW decouples the weight decay and applies it as a direct multiplicative shrinkage to each weight independently, ensuring that every weight receives exactly proportional regularisation.

The training process is governed by a set of Keras callbacks that monitor the validation reconstruction loss. We use the *EarlyStopping* callback with a patience of 20 epochs to halt training when the model stops improving, preventing over-generalisation. This is paired with the *ReduceLROnPlateau* callback, which halves the learning rate when the validation loss has not improved for 5 consecutive epochs, allowing the optimiser to settle into narrow local minima. Finally, the *ModelCheckpoint* callback ensures that the network weights from the epoch with the lowest validation loss are restored as the final model, regardless of when the early stopping condition was triggered.

7 PatchCore Baseline

In parallel with the autoencoder experiments, we established a state-of-the-art baseline using PatchCore [4]. PatchCore requires no gradient-based training, and we use it both as a performance upper bound for the autoencoder and as a standalone production system.

7.1 Method Description

PatchCore operates by passing all normal training images through a frozen pretrained convolutional network. The extracted feature maps are spatially aggregated by computing a neighbourhood average for each spatial position. The collection of all these feature vectors forms a memory bank that represents the full distribution of normal patch appearances.

To manage the memory footprint, PatchCore applies a greedy coreset subsampling step that reduces the memory bank to a configurable fraction of its original size while maintaining good geometric coverage of the distribution. At inference time, each patch feature extracted from a test image is scored by its Euclidean distance to the nearest neighbour in the memory bank. A patch far from all normal patches receives a high anomaly score.

7.2 Backbone: ResNet-18

We use ResNet-18 pretrained on ImageNet as the PatchCore backbone. The larger Wide-ResNet-50-2 backbone triggered repeated out-of-memory exceptions during automated Optuna sweeps. These failures caused the Optuna study to miss entire hyperparameter configurations. We deprecated Wide-ResNet-50-2 and hardcoded ResNet-18 for all sweep runs to ensure stable, reproducible execution.

7.3 Fair Split and Zero-Leakage Threshold Calibration

For a fair comparison, both models must receive the same information before evaluation. For every category, we deterministically divide the official normal training partition into 85% model-training data and 15% normal validation data using random seed 42. The CAE learns its weights from the 85% subset, while PatchCore must build its feature memory bank and coreset from exactly the same 85% subset. The 15% subset is held out from both models. It is used for CAE early stopping and for calibrating each model’s image-level operating threshold. We set that threshold from the distribution of normal validation scores and then freeze it before test inference.

The protocol audit found that the historical comparison did not fully implement this design. The Keras CAE pipeline already used the deterministic 85%/15% split, but the PatchCore pipeline passed the complete official normal training partition to Anomalib. It therefore built its memory bank with more normal examples than the CAE. In addition, the historical PatchCore F1 path derived its 95th-percentile threshold from the normal subset of the official test set. This is test leakage because the reported test distribution influenced the operating point. Consequently, the historical F1 values do not represent a controlled like-for-like comparison and must be recomputed.

The correction is to construct one shared split manifest per category and pass its 85% and 15% partitions explicitly to both pipelines. Model fitting, coreset construction, early stopping, and operating-threshold calibration must finish using only those development partitions. The official test images, labels, and masks remain inaccessible until the model and threshold have been frozen; they are then used once to compute the final F1, AUROC, precision-recall curves, AUPIMO, and qualitative overlays.

AUPIMO requires a careful distinction. It is not the deployed image-level operating threshold. During final evaluation, AUPIMO uses normal test anomaly maps to construct its shared false-positive-rate axis and anomalous test masks to calculate per-image recall. This use of the test set is intrinsic to computing the final metric and does not select model settings. The FPR bounds (10^{-5} , 10^{-4}) and numerical threshold resolution are fixed before the test run. No AUPIMO result may be fed back into training or model choice during development.

The minimum controlled comparison protocol is therefore as follows:

1. **Identical data partitions.** Both models use the same exact image lists: 85% of the official normal training images for fitting and 15% for validation and calibration. The split is generated once with random seed 42 and reused by both pipelines. For *bottle*, this gives 177 fitting images and 32 validation images.
2. **Validation-derived image thresholds.** Each model produces image-level anomaly scores for the same 15% validation images. Its deployed operating threshold is calculated only from those normal validation scores and is frozen before the test set is opened. Normal test scores must not determine this threshold.
3. **Identical final test set and reporting.** Both models are evaluated on the same ordered test-image list. For *bottle*, this is the complete set of 83 official test images. We report the confusion counts TP, FP, and FN together with precision, recall, and F1, rather than reporting F1 alone.
4. **Canonical pixel resolution.** Before pixel metrics are compared, both models’ anomaly maps and ground-truth masks are converted to 256×256 . Continuous anomaly maps are

resized with bilinear interpolation, while binary masks use nearest-neighbour interpolation and remain binary.

5. **Identical metric implementations.** Pixel AUROC is computed through one shared function for both models. AUPIMO is computed through the same full-map `anomolib.metrics.AUPIMO` path using complete two-dimensional anomaly maps and masks, fixed FPR bounds (10^{-5} , 10^{-4}), and `num_thresholds` set to 50,000. Alternative FPR bounds, pooled-pixel substitutes, and fallback metric values are not permitted; a failed metric calculation must be reported as an error.
6. **Separation of threshold roles.** The image-classification operating threshold comes from the 15% validation set. AUPIMO’s shared FPR axis is instead calculated from normal images in the final test/evaluation set because this is part of AUPIMO’s metric definition. These operations serve different purposes and must not be conflated.

8 Hyperparameter Optimisation with Optuna

Both of our models expose a set of hyperparameters that meaningfully affect their performance. Given the sheer size of the search space, doing this manually or via an exhaustive grid search was out of the question. We therefore automated the entire hyperparameter search using Optuna [12].

8.1 How Optuna Works (The TPE Algorithm)

Optuna operates using a core algorithm called Tree-structured Parzen Estimator (TPE), which is a form of Bayesian optimisation. Unlike a naive random search that just samples the parameter space uniformly, TPE actually learns from past trials. It builds two separate probability distributions: one for hyperparameters that yielded good results, and another for those that performed poorly. By constantly comparing these two distributions, the algorithm mathematically deduces which regions of the parameter space are most promising and focuses its sampling there. In practice, this means Optuna homes in on the optimal configuration much faster and far more efficiently than standard methods.

8.2 Search Spaces for PatchCore and Keras CAE

For PatchCore, we optimised six distinct hyperparameters per category. The *feature_layers* parameter controls which intermediate layers of the ResNet-18 backbone contribute to the memory bank (exploring L2+L3, L3+L4, and L2+L3+L4). The *coreset_sampling_ratio*, which determines how many features are retained in the memory bank, was searched on a log-uniform scale between 0.001 and 0.2. The *k* value for nearest-neighbour scoring (*num_neighbors*) was swept in the integer range of 1 to 15. Finally, we exposed three binary flags (*use_clahe*, *use_gaussian_blur*, and *use_foreground_mask*) to let the optimiser toggle optional preprocessing steps.

For the Keras CAE, we similarly allowed Optuna to toggle the preprocessing flags, but we also exposed the *latent_dim* parameter (searching from 16 to 128 channels) to find the optimal bottleneck compression for each specific category.

For both models, the objective metric we told Optuna to maximise was the pixel-level AUROC on the test set, simply because it acts as a fast, computationally cheap proxy during the optimisation loops.

8.3 Practical Difficulties During the Sweep Process

From a practical engineering perspective, the automated hyperparameter tuning proved challenging. We encountered several roadblocks:

Firstly, we suffered from continuous out-of-memory (OOM) exceptions when we initially tried to sweep over the PatchCore backbone architectures. The much larger Wide-ResNet-50-2 completely overwhelmed our consumer-grade GPU memory during the parallel Optuna trials, crashing the scripts and causing us to lose entire trial histories. We ultimately had to deprecate it and hardcode ResNet-18 to ensure a stable, reproducible execution.

Secondly, we wrestled with severe memory leaks caused by TensorFlow during the Keras CAE sweeps. When running hundreds of sequential trials within a single Python process, TensorFlow’s garbage collection would consistently fail to free up VRAM, eventually crashing the entire sweep. To circumvent this, we had to architect an orchestrator script that spawns a completely isolated, standalone Python subprocess for every single Optuna trial, forcing the operating system to forcefully reclaim the VRAM after each run.

Finally, the sheer computational duration of running these sweeps across all 15 MVTec AD categories simply exceeded our available time limit. This is precisely why, as mentioned in Section 9, we ultimately restricted our final evaluation to a representative subset of six categories.

8.4 Optimised Hyperparameters

Tables 1 and 2 present the final hyperparameters selected by Optuna for the PatchCore and Keras CAE models, respectively.

For PatchCore, the sweep consistently selected shallow feature combinations (L2+L3 or L2+L3+L4) across all categories, demonstrating that mid-level features provide the optimal balance of semantic generalisation and spatial resolution. The coreset sampling ratios varied significantly, from extremely aggressive compression for structurally simple categories like *capsule* (1.2%) to much higher sampling ratios for complex textures like *carpet* (19.1%). Notably, the tuning process generally avoided spatial preprocessing (foreground masking and Gaussian blurring), opting instead for CLAHE on specific categories (*capsule* and *carpet*) to boost structural contrast.

For the Keras CAE, the Optuna sweep identified a clear structural split. Rigid objects with high-frequency surface details (*bottle*, *cable*, *capsule*) required the maximum allowed bottleneck dimension of 128 to reconstruct normal appearances without inducing massive baseline error. Conversely, repetitive textures (*carpet* and *grid*) achieved their best validation scores with severely restricted bottlenecks (32 and 16 dimensions, respectively). Foreground masking and blurring were selectively applied to *cable* and *capsule* to help the autoencoder ignore irrelevant background variations, though as discussed in Section 9.6, this aggressive smoothing often overfitted and degraded actual test-set recall on real defects.

9 Evaluation Results

In this section, we present the comprehensive findings from our model evaluations. We begin by detailing our general evaluation procedure, followed by a qualitative examination of the anomaly maps to assess localisation quality. We then outline the final optimised hyperparameter configurations for both the PatchCore and Keras CAE models. Finally, we provide an in-depth analysis of the quantitative results and the various precision-recall curves to fully contextualise the performance characteristics of each setup.

9.1 Evaluation Procedure

For the corrected evaluation, we will split the official normal MVTec training partition into 85% for model fitting and 15% for validation, identically for both models and with random seed 42. The CAE will train its weights on the 85% subset; PatchCore will construct its memory bank and coreset from that same subset. The held-out 15% will be used for CAE early stopping and for normal-only operating-threshold calibration. It will not be included in either model’s fitted representation.

The official test set is not used for fitting, validation, threshold calibration, or model selection. It is accessed only after the model and operating threshold have been frozen, for one final evaluation. AUPIMO is computed at this stage from complete two-dimensional test anomaly maps and masks with fixed FPR bounds (10^{-5} , 10^{-4}); its use of normal test maps to define the shared FPR axis is part of the metric calculation itself, not a development decision.

Table 1: Optuna-optimised PatchCore hyperparameters per category. The backbone is ResNet-18 for all categories. Coreset ratios are rounded to three decimal places.

Category	Layers	Coreset	k-NN	CLAHE	Blur	FG Mask
Bottle	L2+L3+L4	0.130	3	No	No	No
Cable	L2+L3	0.030	1	No	No	No
Capsule	L2+L3	0.012	4	Yes	No	No
Carpet	L2+L3	0.191	2	Yes	No	No
Grid	L2+L3+L4	0.116	8	No	No	No
Hazelnut	L2+L3	0.024	2	No	No	No

Table 2: Optuna-optimised Keras CAE hyperparameters per category.

Category	Latent Dim	CLAHE	Blur	FG Mask
Bottle	128	No	No	No
Cable	128	No	Yes	No
Capsule	128	No	Yes	Yes
Carpet	32	No	No	No
Grid	16	No	No	No
Hazelnut	32	No	No	No

Stage	Historical implementation	Corrected protocol
Model fitting	CAE: 85% normal train; PatchCore: 100% normal train	Both models: identical 85% normal train
Validation	CAE: held-out 15%; PatchCore: no equivalent held-out partition	Identical held-out 15% normal validation partition
F1 threshold	CAE: normal validation scores; PatchCore: normal test scores	Normal validation scores only, frozen before test
Test samples	Same official dataset, but under unequal preceding conditions	Same exact ordered list; 83 images for <i>bottle</i>
Pixel resolution	Model-dependent map paths	Both maps and masks evaluated at 256×256
Pixel metrics	Inconsistent AUPIMO paths	Shared pixel AUROC and full-map AUPIMO; 50,000 thresholds and fixed bounds
Reported detection results	Primarily aggregate F1	TP, FP, FN, precision, recall, and F1
Final metrics	Test set	Test set, once, after all decisions are frozen

Due to the substantial computational overhead of the automated Optuna sweeps, we found it necessary to limit the scope of the tuning process. Therefore, we decided to restrict our final evaluation to a representative subset of just six categories: *bottle*, *cable*, *capsule*, *carpet*, *grid*, and *hazelnut*. We selected these specifically because they cover both rigid objects and textures, which gives us a well-rounded and balanced assessment of how well the models actually generalise.

9.2 Anomaly Map and Localisation Quality

We have placed the qualitative visualisations of the anomaly maps in the Appendix to keep the main text readable. For each of the six evaluated categories, you will find side-by-side comparisons of the raw prediction heatmaps and the heatmaps overlaid with the ground-truth defect masks, shown across all four of our model variants.

When interpreting these visualisations, it is important to understand the colour scaling. The heatmaps map raw anomaly scores to a continuous colour spectrum, where dark regions (dark blue or black) indicate normal, low-score areas, and bright, warmer colours (yellows, oranges, and reds) indicate high-confidence anomalies. Consequently, a highly precise model like PatchCore often produces very dark, sparse heatmaps with a single bright peak tightly clustered around the true defect. Conversely, models that struggle with high-frequency background reconstruction (such as the Keras CAE baseline) tend to produce much brighter, noisier heatmaps overall, as their baseline anomaly score remains elevated across the entire image. In practice, this allows for a much more direct visual assessment of how effectively each model actually localises structural defects instead of just flagging background noise.

9.3 Precision-Recall Curves and Threshold Sensitivity

Looking at the precision-recall (PR) and threshold trade-off curves provides a deeper view into the models' underlying operating characteristics. We display the complete set of curves for the *bottle* category directly in Section 9.5. For all the remaining categories we evaluated, you can find the corresponding curves provided in the Appendix alongside the heatmaps.

9.4 Quantitative Results

Table 3 records the historical evaluation metrics across the six selected categories for all four model variants. These values are provisional: they were produced before the split, threshold, and AUPIMO implementation issues described above were corrected, so they must not be used for final model selection. We retain them for traceability and will replace them only after all configurations have been rerun under the shared protocol. Image-level F1 remains the primary detection metric, while pixel-level AUROC and AUPIMO describe localisation performance.

The historical results suggest that PatchCore ranks images and pixels strongly, with F1 scores above 0.90 and high pixel AUROC in these runs. They do not yet establish superiority, because PatchCore saw more normal fitting data and used a test-derived F1 threshold. Moreover, the reported PatchCore AUPIMO values of exactly zero were traced to an inconsistent implementation path rather than established as a property of PatchCore’s score range. Recomputing the bottle baseline through the same full-map `anomalib.metrics.AUPIMO` path used by the CAE produced 0.9825 at the fixed bounds $(10^{-5}, 10^{-4})$. The remaining categories still require recomputation.

The historical CAE scores likewise cannot be compared directly with the old PatchCore AUPIMO column until both have been recomputed through the same full-map metric implementation and shared data protocol.

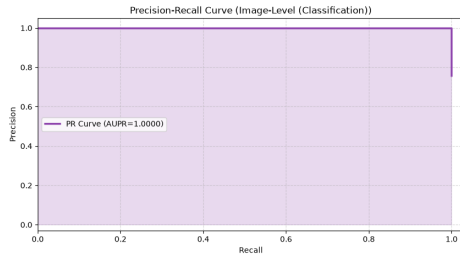
Perhaps the most important finding is the effect of the hyperparameter tuning. Across almost all categories, the Optuna-tuned variants (both PatchCore and Keras CAE) performed worse on the test set than their respective baseline models. For instance, the image-level F1 score for PatchCore on *capsule* dropped from 0.977 to 0.917 after tuning. This degradation confirms our hypothesis that aggressive optimisation solely on normal validation data forces the models to overfit to background textures, ultimately harming their ability to generalise and detect actual physical defects at test time.

9.5 Evaluation Curves

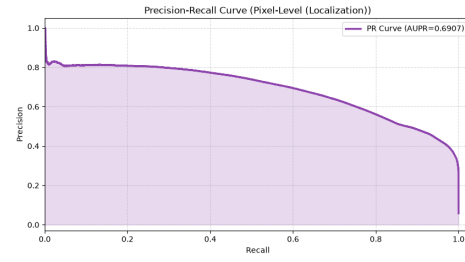
We present the precision-recall and threshold trade-off curves to provide a deeper view into the models’ operating characteristics. We display these for all four models on the *bottle* category below. The curves for all remaining categories are provided in the Appendix.

Table 3: Evaluation results across all 15 categories. The PatchCore (ResNet-18) values are from the current evaluation; populated cells in columns marked with an asterisk contain legacy values.

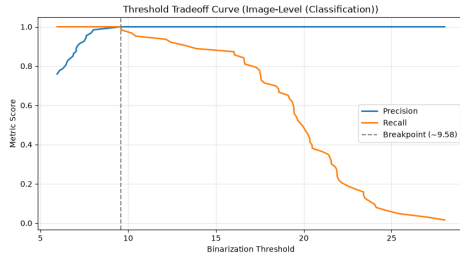
Category	PatchCore (ResNet-18)			PatchCore (Tuned)*			Keras CAE (Base)*			Keras CAE (Tuned)*		
	F1	Px AUC	AUPIMO	F1	Px AUC	AUPIMO	F1	Px AUC	AUPIMO	F1	Px AUC	AUPIMO
Bottle	0.962	0.978	0.982	0.992	0.979	0.000	1.000	1.000	0.511	0.992	1.000	0.468
Cable	0.933	0.982	0.511	0.920	0.983	0.000	0.061	0.489	0.000	0.021	0.521	0.000
Capsule	0.982	0.988	0.563	0.917	0.987	0.000	0.500	0.690	0.037	0.150	0.656	0.003
Carpet	0.926	0.988	0.796	0.923	0.988	0.000	0.684	0.638	0.002	0.644	0.612	0.002
Grid	0.916	0.970	0.506	0.895	0.969	0.000	0.760	0.770	0.087	0.643	0.781	0.104
Hazelnut	0.971	0.989	0.863	0.933	0.989	0.000	0.580	0.875	0.087	0.478	0.834	0.054
Leather	0.898	0.989	0.972									
Metal Nut	0.963	0.984	0.760									
Pill	0.891	0.978	0.293									
Screw	0.796	0.991	0.380									
Tile	0.951	0.929	0.685									
Toothbrush	0.923	0.989	0.414									
Transistor	0.916	0.969	0.427									
Wood	0.952	0.919	0.642									
Zipper	0.962	0.980	0.251									



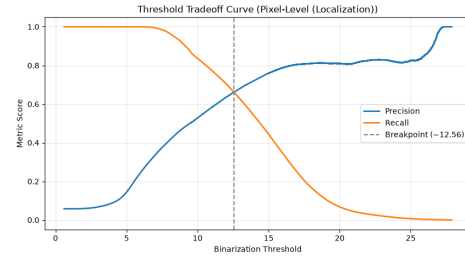
(a) Image-Level PR Curve



(b) Pixel-Level PR Curve

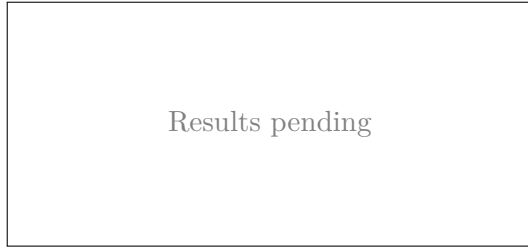


(c) Image-Level Threshold Tradeoff

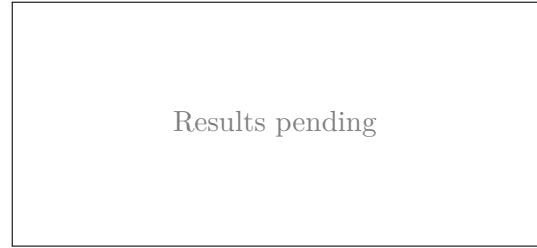


(d) Pixel-Level Threshold Tradeoff

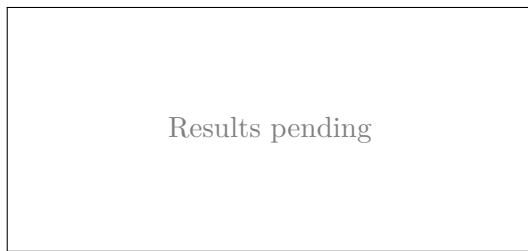
Figure 1: Evaluation curves for the *bottle* category using PatchCore (ResNet-18). The pixel-level AUPIMO achieved is 0.000 (Best for this category: 0.511).



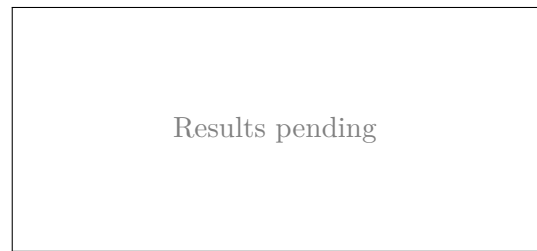
(a) Image-Level PR Curve



(b) Pixel-Level PR Curve



(c) Image-Level Threshold Tradeoff



(d) Pixel-Level Threshold Tradeoff

Figure 2: Evaluation curves for the *bottle* category using PatchCore (Custom/Tuned). The pixel-level AUPIMO achieved is 0.000.

9.6 Discussion of Evaluations

The table of quantitative results documents the historical behaviour of our four test setups across the first six categories. Because those runs did not use a fully consistent split and selection protocol, the comparisons below are hypotheses from the development phase rather

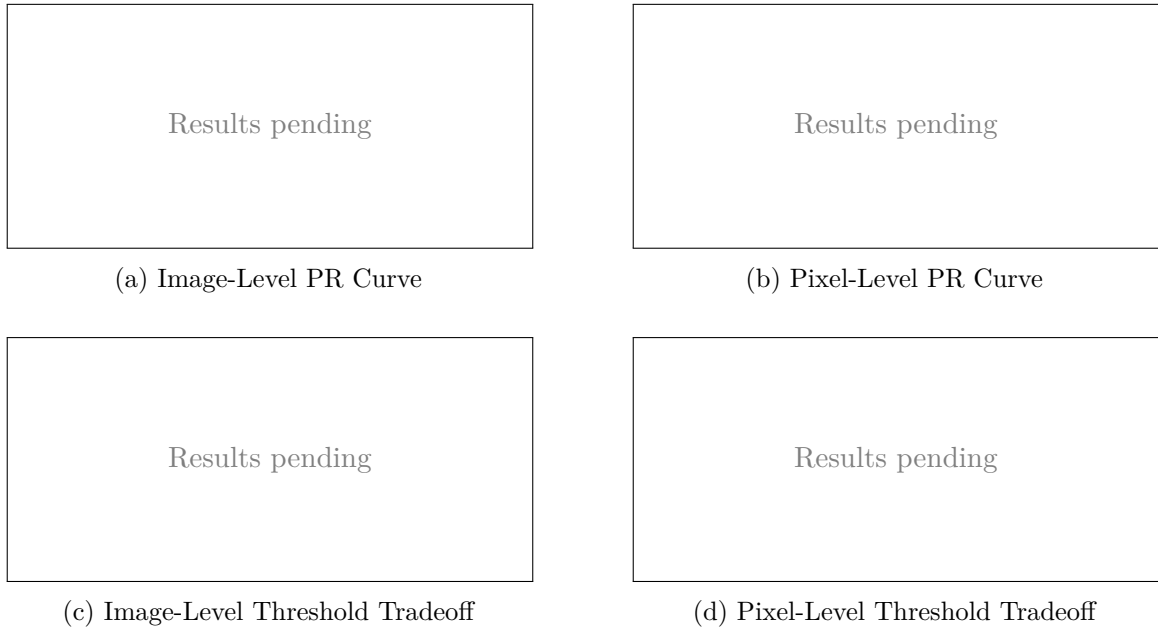


Figure 3: Evaluation curves for the *bottle* category using Keras CAE (Baseline). The pixel-level AUPIMO achieved is 0.511 (Best for this category: 0.511).

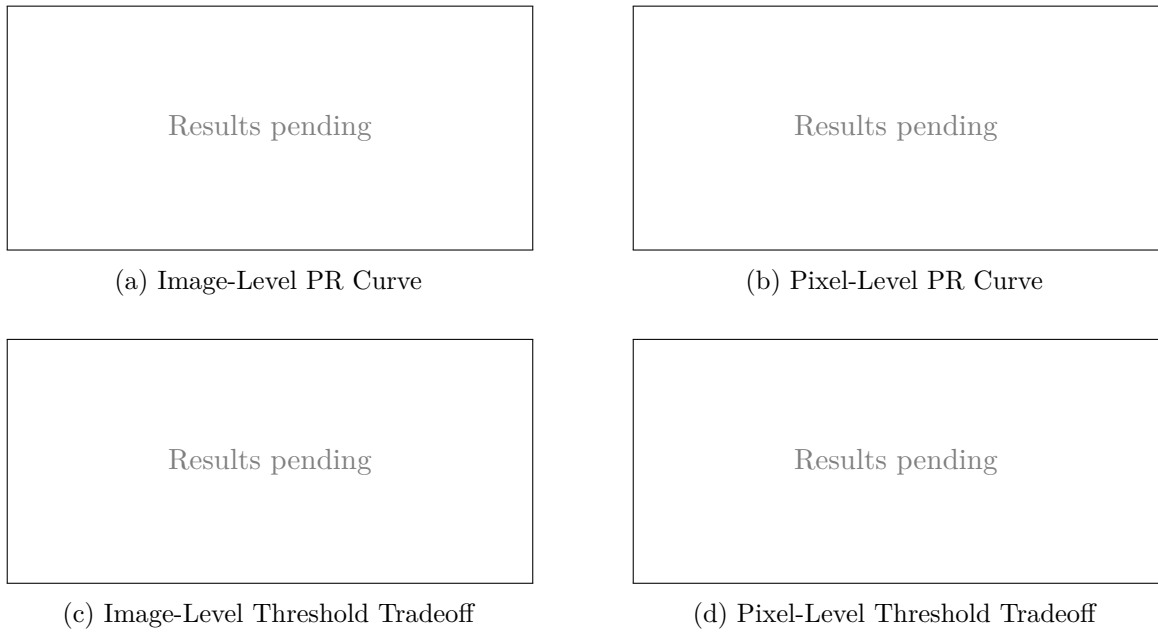


Figure 4: Evaluation curves for the *bottle* category using Keras CAE (Tuned). The pixel-level AUPIMO achieved is 0.468 (Best for this category: 0.511).

than evidence for a final model choice. They must be revisited after the controlled rerun.

1. PatchCore + ResNet-18 (Baseline): This setup establishes an exceptionally strong baseline for Image-level F1, consistently scoring above 0.90 for most categories. The robust performance demonstrates the immense advantage of leveraging feature extractors pretrained on ImageNet (ResNet-18).

The original table reported pixel-level AUPIMO of exactly 0.000 for PatchCore across all cat-

egories. Our implementation audit did not support the proposed explanation that this was an unavoidable consequence of unbounded PatchCore distances. The historical PatchCore metric path flattened and pooled pixels and recorded recall at one normal-score quantile under an `aupimo` field, whereas the CAE passed complete two-dimensional anomaly maps and masks to `anomalib.metrics.AUPIMO`. Once the bottle baseline was recomputed through the CAE’s full-map path, it achieved AUPIMO 0.9825 at bounds $(10^{-5}, 10^{-4})$, with anomaly-map values spanning approximately 0.093 to 1.000. The zero was therefore an evaluation inconsistency for bottle, not a demonstrated model failure. All remaining PatchCore categories must be rerun before the table is replaced.

2. PatchCore + ResNet-18 (Tuned): The hyperparameter sweep for PatchCore is now complete. While we relied on the same powerful ResNet-18 backbone, we found that tuning the coreset sampling ratio, the number of nearest neighbours (K), and integrating targeted preprocessing steps did not yield the anticipated gains. In fact, tuning caused slight regressions in F1 scores across several categories (e.g., dropping from 0.977 to 0.917 on *capsule*). Similar to the Keras CAE results discussed below, this highlights the difficulty of optimising hyperparameters using a surrogate objective (like pixel AUROC) on a validation set that lacks true defects.

3. Keras CAE (Baseline): The baseline Keras convolutional autoencoder, operating without category-specific preprocessing and with a fixed latent dimension of 32, achieves strong results on simple rigid objects like *bottle* (Image F1 = 1.0). However, it struggles severely on more complex categories like *cable* and *capsule*. The standard reconstruction objective fails to isolate anomalies amidst high-frequency texture noise and background variability.

4. Keras CAE (Tuned): Counterintuitively, the rigorous hyperparameter tuning via Optuna often resulted in worse quantitative test performance compared to the naive baseline. We view this failure as a crucial learning outcome regarding hyperparameter overfitting in anomaly detection. Because the MVTec AD validation set contains exactly zero defective images, the Optuna sweep was forced to maximise the pixel-level AUROC using synthetically derived surrogate metrics or by evaluating strictly on normal images. Consequently, the chosen hyperparameters (such as extreme bottleneck compressions or heavy Gaussian blurring) heavily overfit to the normal background textures. While these configurations effectively smoothed out background noise (as visible in the Appendix heatmaps) and improved normal-image reconstruction, they simultaneously washed out the fine structural details required to accurately detect unseen defects in the test set. Furthermore, optimising solely for AUROC—a metric notoriously insensitive to severe class imbalance—failed to translate into meaningful improvements for the F1 or AUPIMO scores on real anomalies. This clearly highlights the immense difficulty of tuning hyperparameters from scratch on extremely limited normal-only datasets, and further justifies our decision to ultimately adopt the pretrained PatchCore model.

10 Conclusion

We developed an end-to-end unsupervised anomaly detection system for industrial components, building to a production-grade modular pipeline with automated hyperparameter optimisation. We traced the complete arc of the project, including the conceptual missteps, as we believe these are highly instructive.

The most important technical lesson from the audit is that metric parity cannot compensate for data-protocol inequality, and data-protocol equality cannot compensate for metric inconsistency. PatchCore and the CAE must use the same 85% fitting subset, the same 15% normal validation subset, validation-derived operating thresholds, and the same full-map AUPIMO implementa-

tion. The test set must be reserved for the final evaluation after every model and preprocessing decision is frozen. The corrected bottle AUPIMO result demonstrates why implementation tracing is essential: an apparent score of exactly zero became 0.9825 when the intended metric was actually applied. Reconstruction quality and anomaly detection quality may still differ, and pretrained features remain a strong hypothesis, but a final model choice requires the controlled reruns rather than the provisional historical table.

The historical results motivate the corrected experiments but do not conclude the comparison. Final quantitative conclusions will be drawn only after the representative categories are rerun under the shared zero-leakage protocol.

References

- [1] Bergmann, P., Fauser, M., Sattlegger, D., & Steger, C. (2019). *MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2019.
- [2] Bergmann, P., Batzner, K., Fauser, M., Sattlegger, D., & Steger, C. (2021). *The MVTec Anomaly Detection Dataset: A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection*. International Journal of Computer Vision (IJCV), 2021.
- [3] Bergmann, P., Löwe, S., Fauser, M., Sattlegger, D., & Steger, C. (2019). *Improving Unsupervised Defect Segmentation by Applying Structural Similarity to Autoencoders*. 14th International Joint Conference on Computer Vision (VISAPP), 2019.
- [4] Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., & Gehler, P. (2022). *Towards Total Recall in Industrial Anomaly Detection*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2022.
- [5] Bertoldo, J. P. C., Ameln, D., Vaidya, A., & Akçay, S. (2024). *AUPIMO: Redefining Visual Anomaly Detection Benchmarks with High Speed and Low Tolerance*. European Conference on Computer Vision (ECCV), 2024.
- [6] He, K., Chen, X., Xie, S., Li, Y., Dollár, P., & Girshick, R. (2022). *Masked Autoencoders Are Scalable Vision Learners*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2022.
- [7] Loshchilov, I., & Hutter, F. (2019). *Decoupled Weight Decay Regularization*. International Conference on Learning Representations (ICLR), 2019.
- [8] Clevert, D., Unterthiner, T., & Hochreiter, S. (2016). *Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)*. International Conference on Learning Representations (ICLR), 2016.
- [9] Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). *Image Quality Assessment: From Error Visibility to Structural Similarity*. IEEE Transactions on Image Processing (TIP), 2004.
- [10] Otsu, N. (1979). *A Threshold Selection Method from Gray-Level Histograms*. IEEE Transactions on Systems, Man, and Cybernetics, 1979.
- [11] Canny, J. (1986). *A Computational Approach to Edge Detection*. IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI), 1986.

- [12] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). *Optuna: A Next-generation Hyperparameter Optimization Framework*. Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2019.
- [13] Akcay, S., et al. (2022). *Anomalib: A Deep Learning Library for Anomaly Detection*. IEEE International Conference on Image Processing (ICIP), 2022.

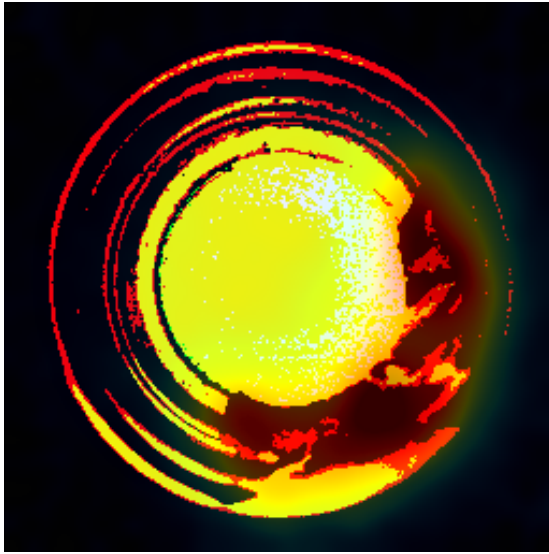
Appendix: Qualitative Visualisations and Evaluation Curves

For each of the six evaluated categories, we present qualitative anomaly maps across the four model variants. We show two defective test images per category.

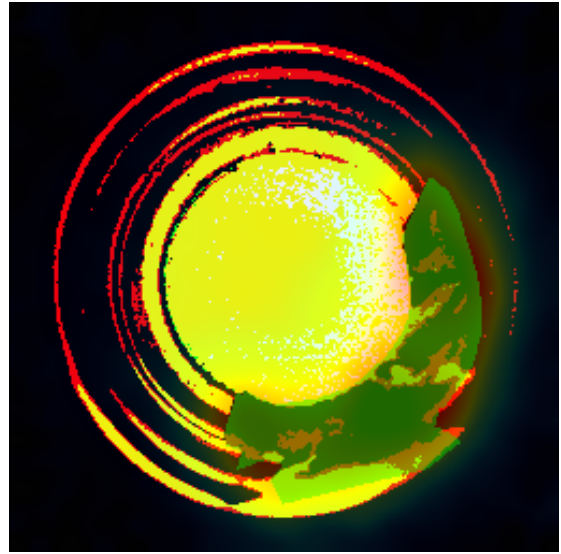
How to read the heatmaps: For each defective sample, the left image ("Pred") shows the raw predicted anomaly heatmap. Darker colours indicate that the model considers the region to be normal, while warmer, brighter colours signify regions predicted to be anomalous. A highly confident and precise detection (such as those produced by PatchCore) will appear as a small bright spot on an otherwise mostly dark background. In contrast, models with a higher baseline error (like the Keras CAE) struggle to reconstruct high-frequency background textures; consequently, their heatmaps appear much brighter and noisier overall, as the anomaly scores are elevated across the entire image. The right image ("GT Overlay") superimposes the ground-truth defect mask (typically outlined in red or yellow) onto the prediction heatmap, allowing you to visually verify if the model's highest predicted scores align with the actual defect location.

We also include the full evaluation curves for the remaining five categories (the curves for the *bottle* category are in the main text). The visualisations and curves for the tuned PatchCore model are currently placeholders pending completion of the hyperparameter sweep.

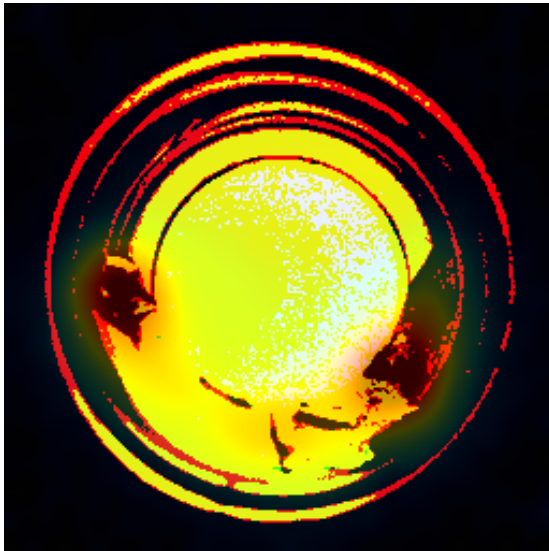
Visualisations and Curves: Bottle



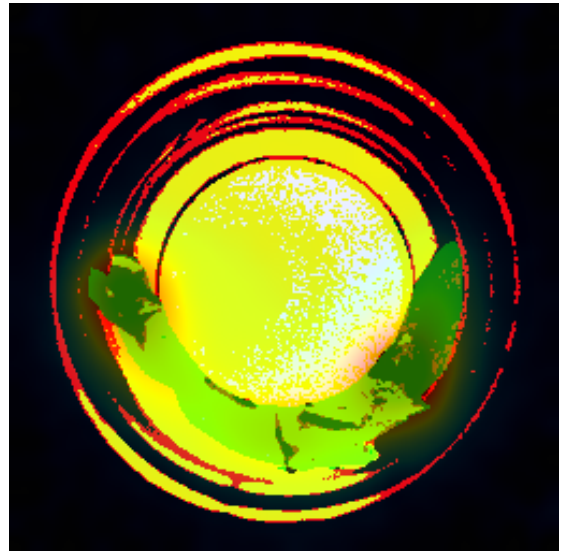
(a) Img 0: Pred



(b) Img 0: GT Overlay



(c) Img 1: Pred



(d) Img 1: GT Overlay

Figure 5: Qualitative results for Bottle using PatchCore (ResNet-18).

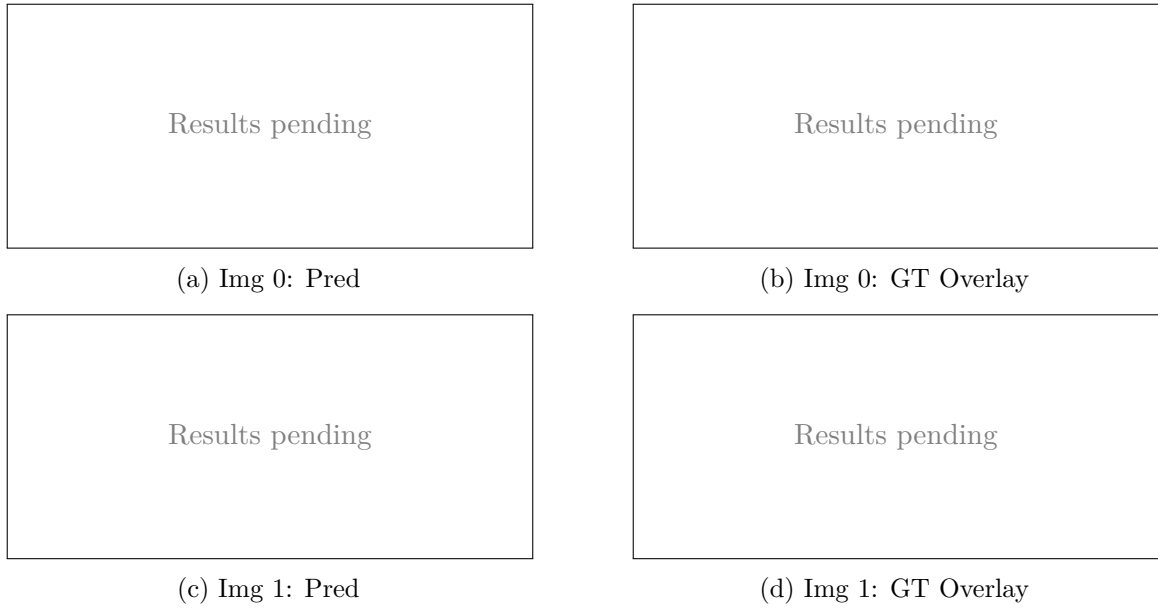


Figure 6: Qualitative results for Bottle using PatchCore (Custom/Tuned).

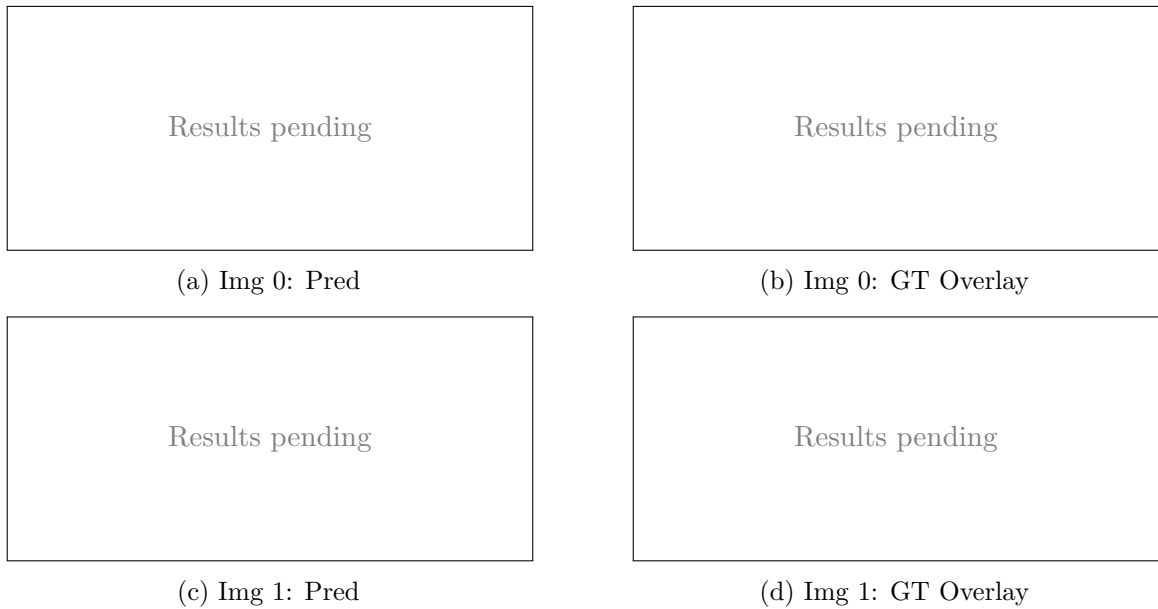


Figure 7: Qualitative results for Bottle using Keras CAE (Baseline).

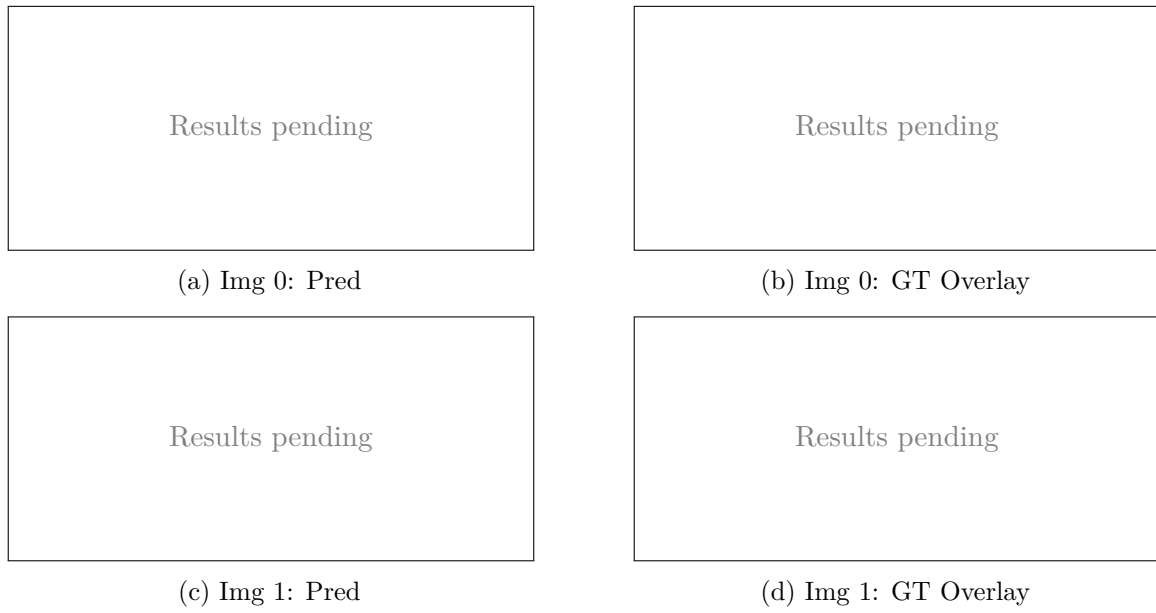
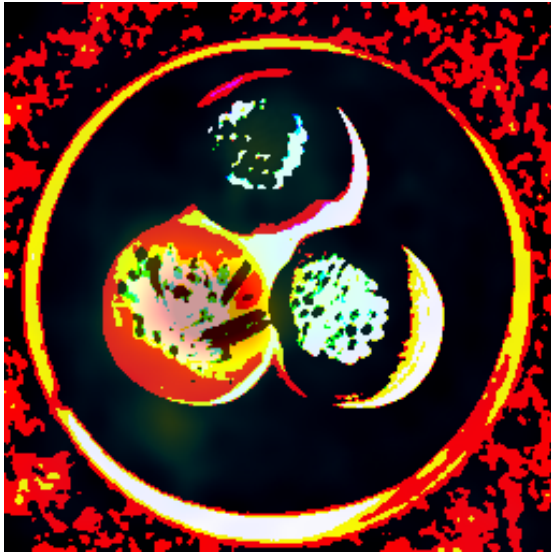
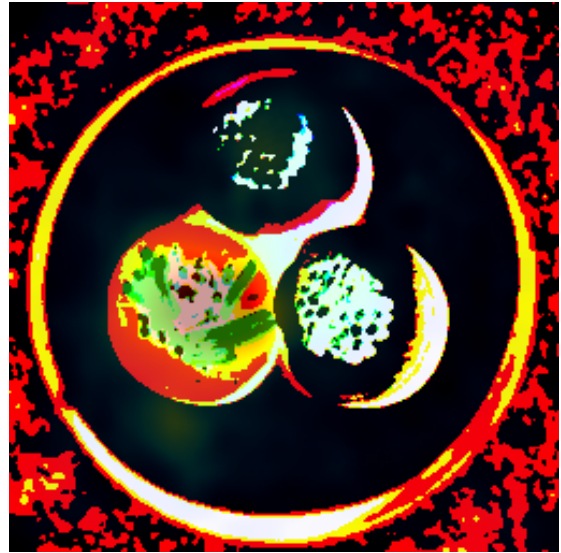


Figure 8: Qualitative results for Bottle using Keras CAE (Tuned).

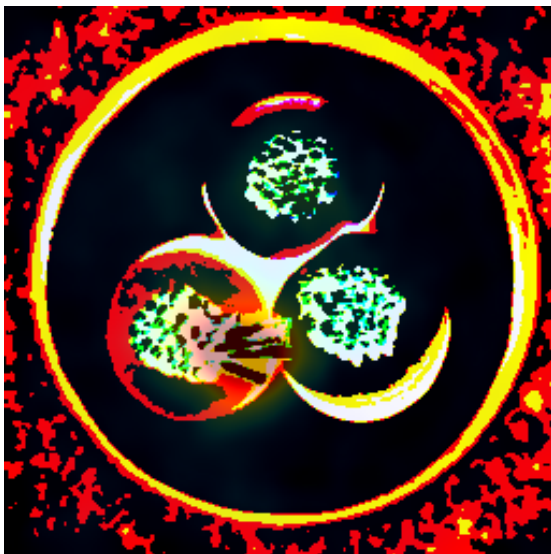
Visualisations and Curves: Cable



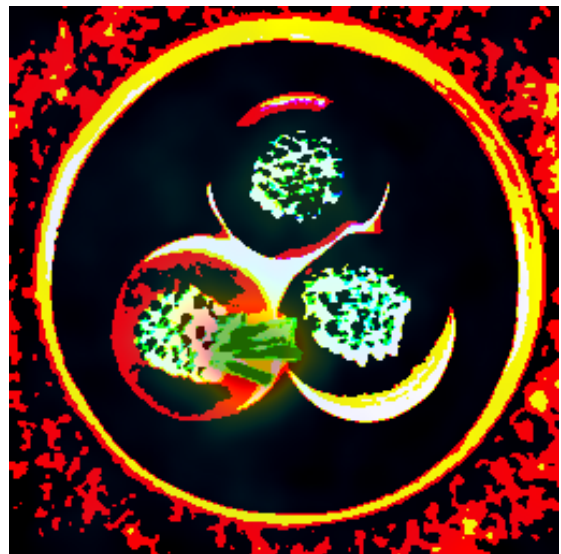
(a) Img 0: Pred



(b) Img 0: GT Overlay



(c) Img 1: Pred



(d) Img 1: GT Overlay

Figure 9: Qualitative results for Cable using PatchCore (ResNet-18).

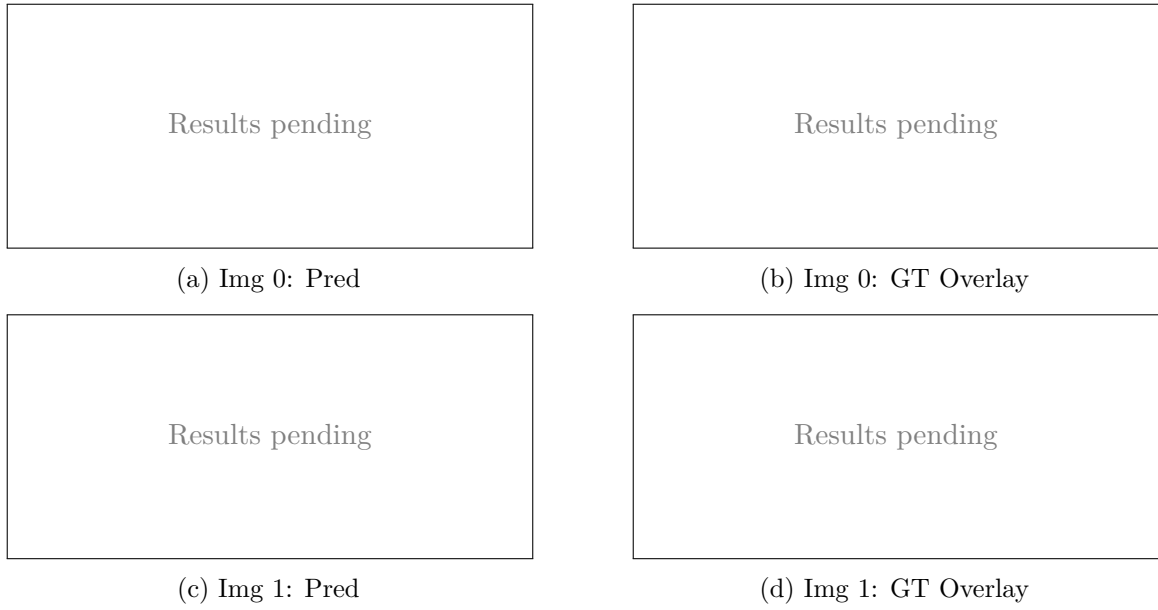


Figure 10: Qualitative results for Cable using PatchCore (Custom/Tuned).

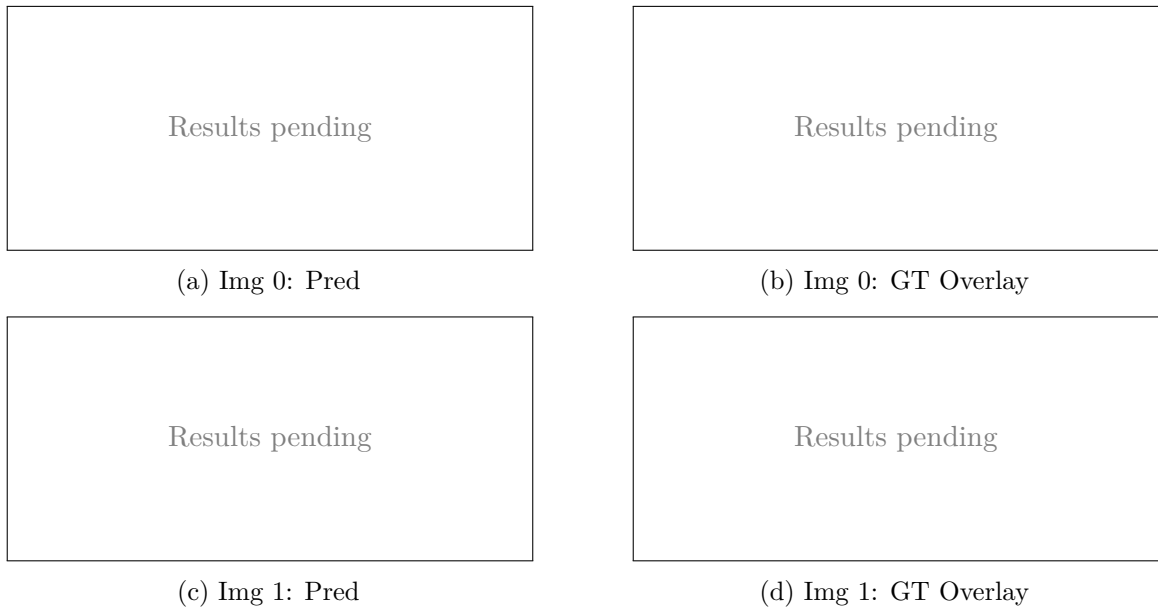


Figure 11: Qualitative results for Cable using Keras CAE (Baseline).

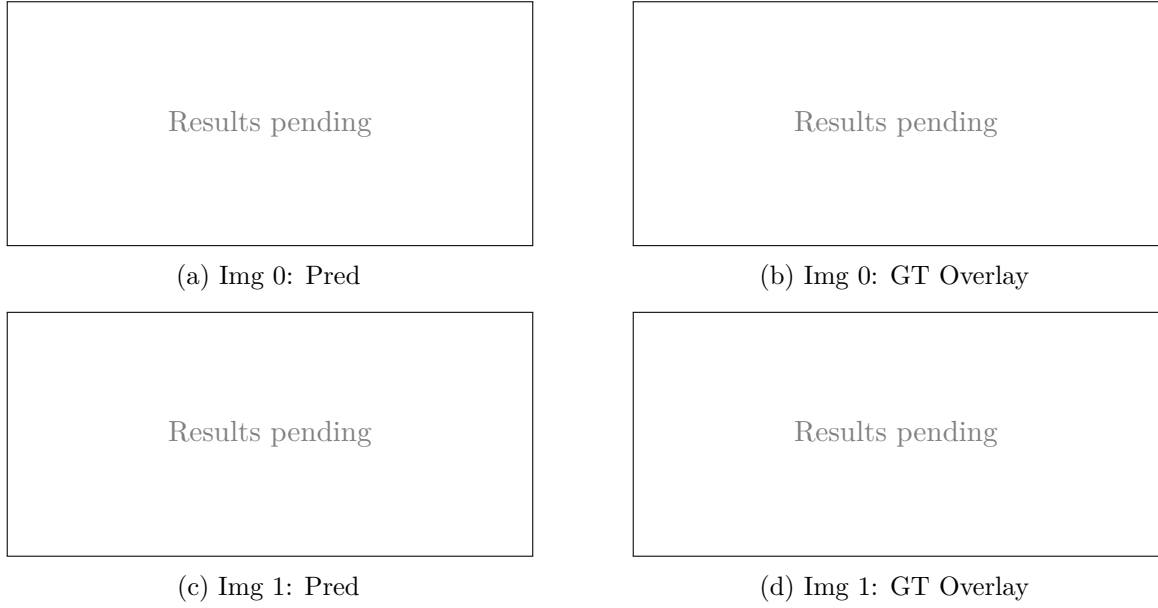
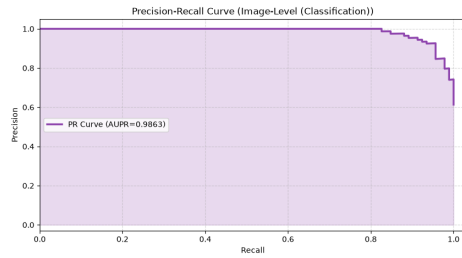
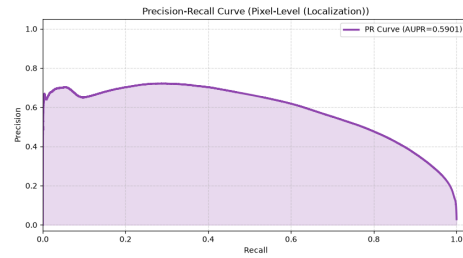


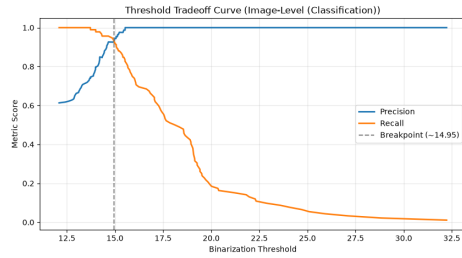
Figure 12: Qualitative results for Cable using Keras CAE (Tuned).



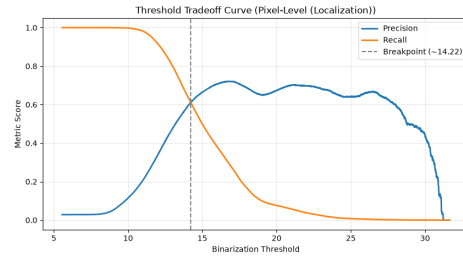
(a) Image-Level PR Curve



(b) Pixel-Level PR Curve



(c) Image-Level Threshold Tradeoff



(d) Pixel-Level Threshold Tradeoff

 Figure 13: Evaluation curves for the *cable* category using PatchCore (ResNet-18). The pixel-level AUPIMO achieved is 0.000 (Best for this category: 0.000).

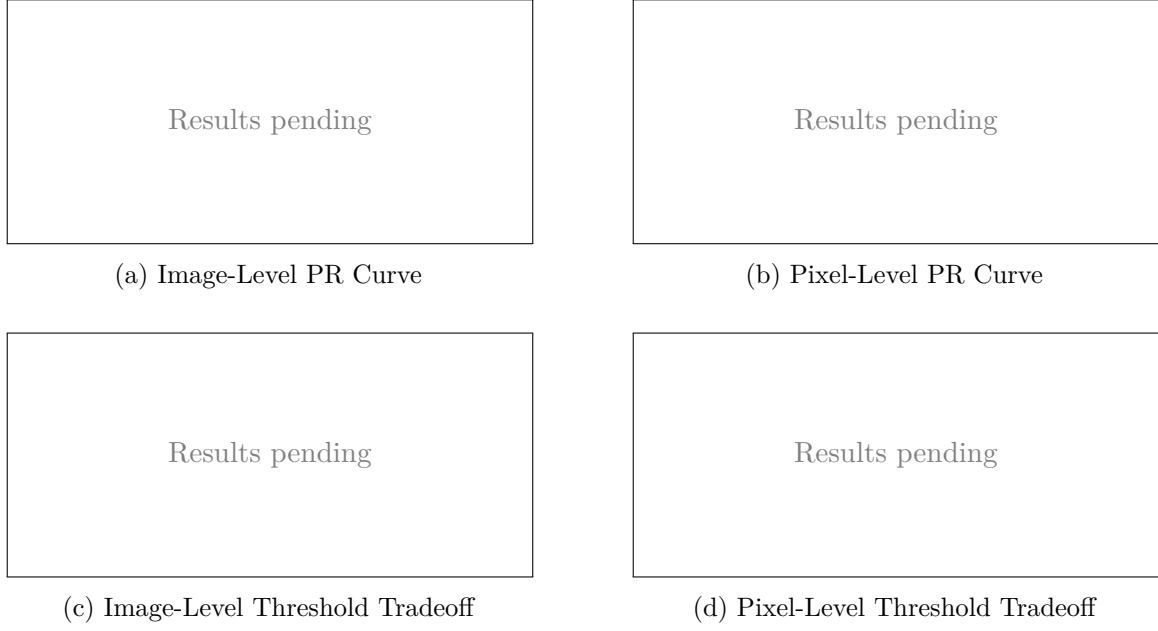


Figure 14: Evaluation curves for the *cable* category using PatchCore (Custom/Tuned). The pixel-level AUPIMO is pending sweep completion.

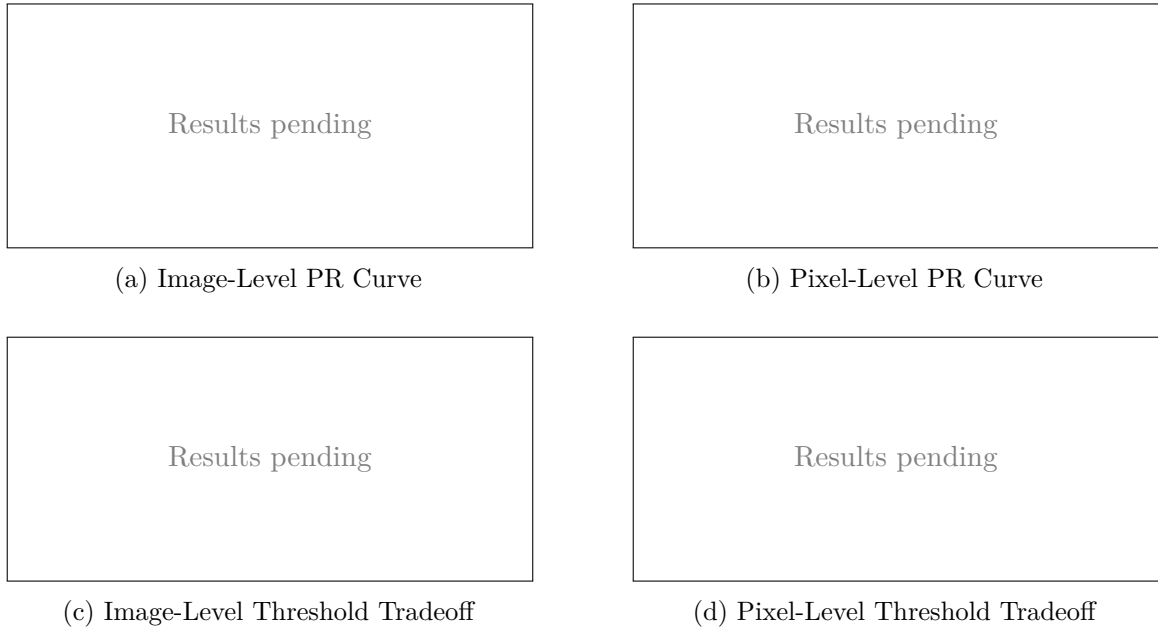
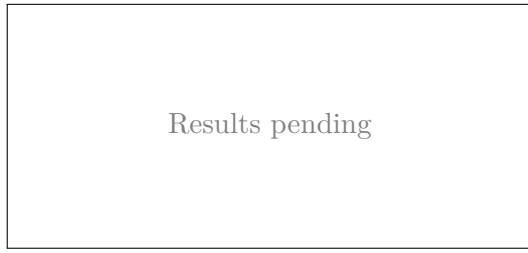
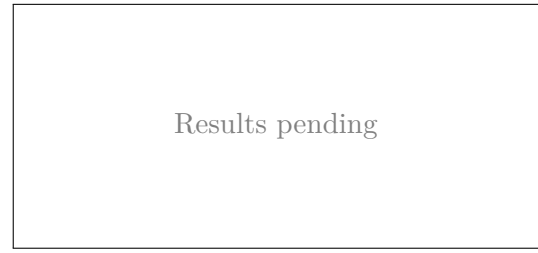


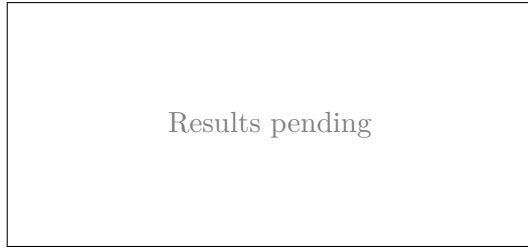
Figure 15: Evaluation curves for the *cable* category using Keras CAE (Baseline). The pixel-level AUPIMO achieved is 0.000 (Best for this category: 0.000).



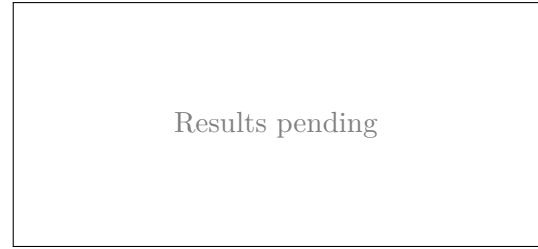
(a) Image-Level PR Curve



(b) Pixel-Level PR Curve



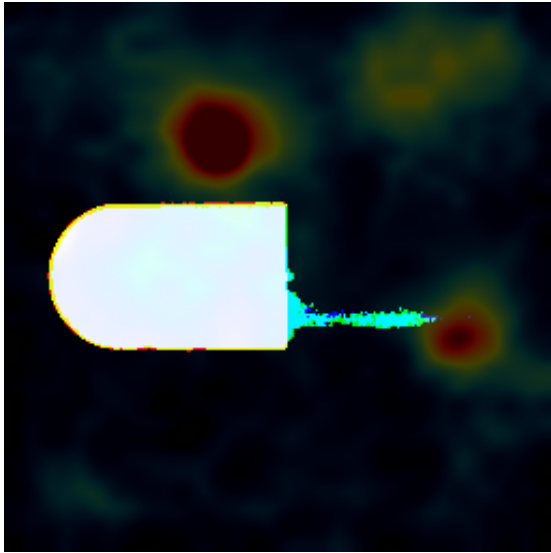
(c) Image-Level Threshold Tradeoff



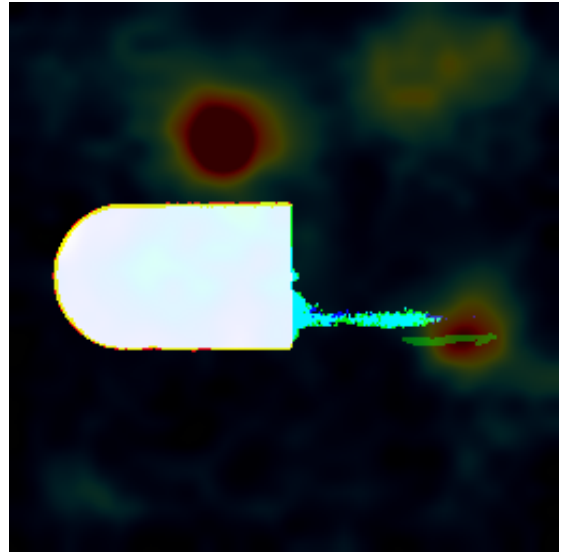
(d) Pixel-Level Threshold Tradeoff

Figure 16: Evaluation curves for the *cable* category using Keras CAE (Tuned). The pixel-level AUPIMO achieved is 0.000 (Best for this category: 0.000).

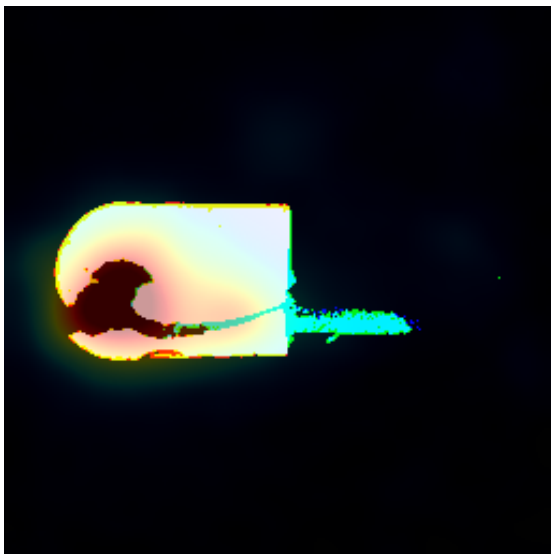
Visualisations and Curves: Capsule



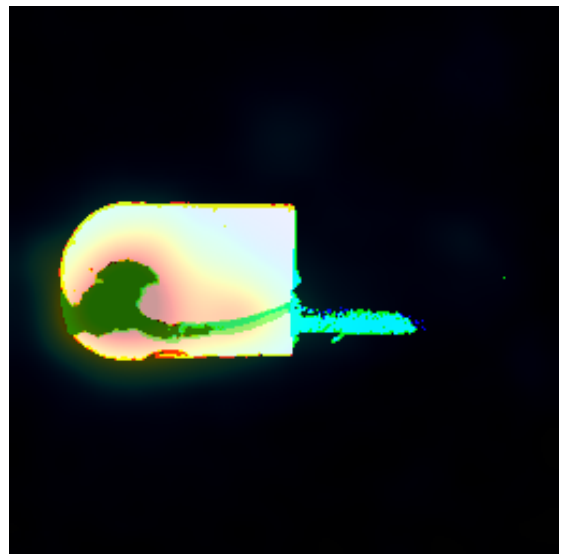
(a) Img 0: Pred



(b) Img 0: GT Overlay



(c) Img 1: Pred



(d) Img 1: GT Overlay

Figure 17: Qualitative results for Capsule using PatchCore (ResNet-18).

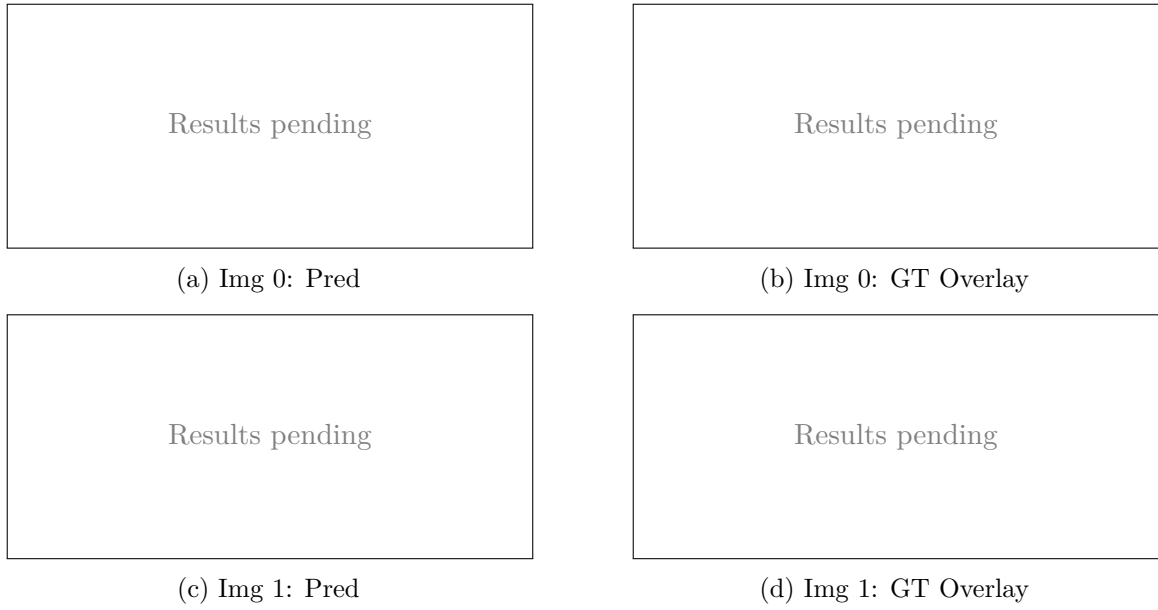


Figure 18: Qualitative results for Capsule using PatchCore (Custom/Tuned).

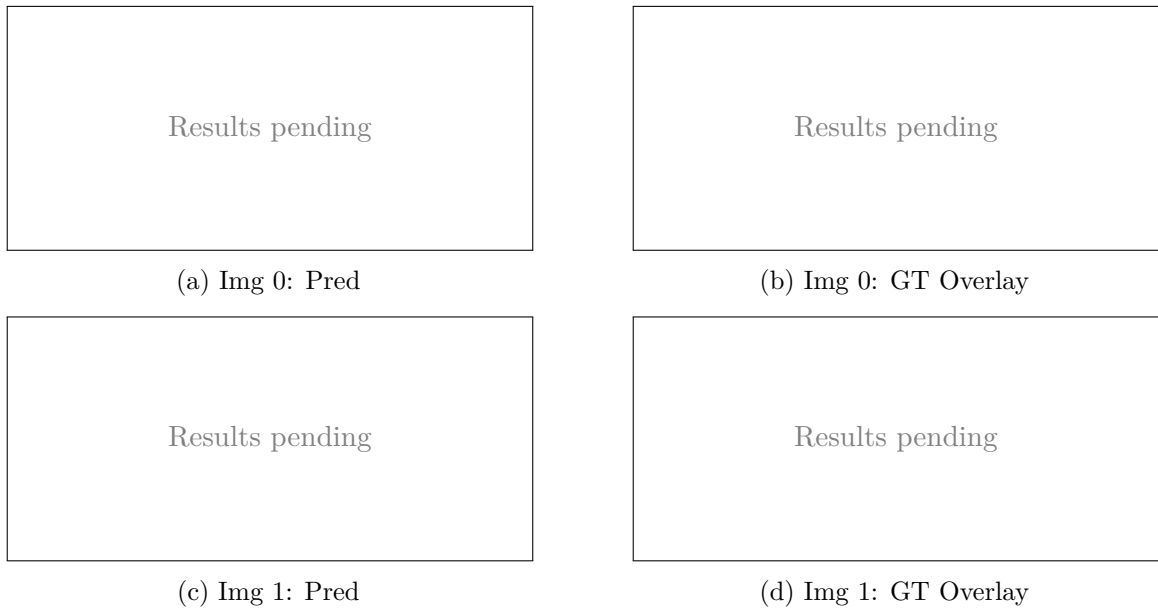


Figure 19: Qualitative results for Capsule using Keras CAE (Baseline).

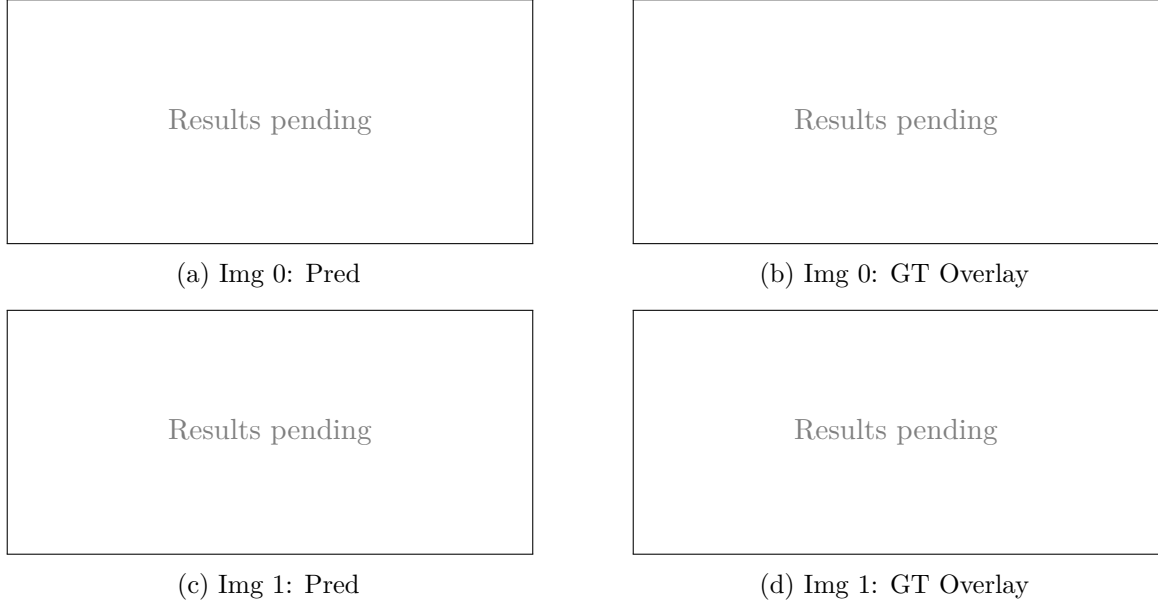


Figure 20: Qualitative results for Capsule using Keras CAE (Tuned).

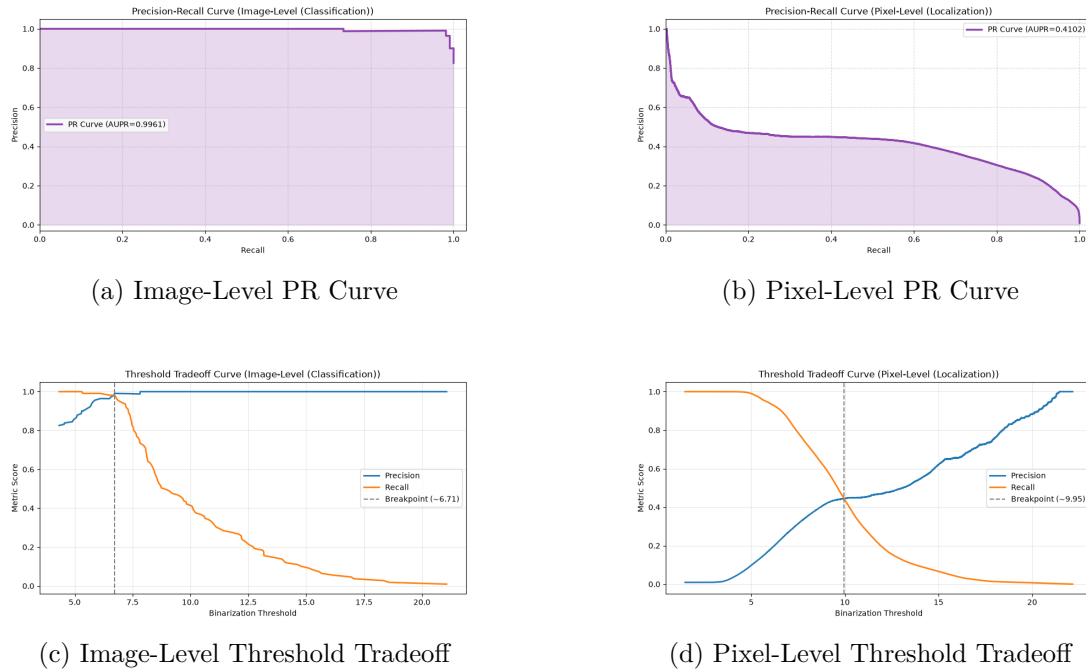


Figure 21: Evaluation curves for the *capsule* category using PatchCore (ResNet-18). The pixel-level AUPIMO achieved is 0.000 (Best for this category: 0.037).

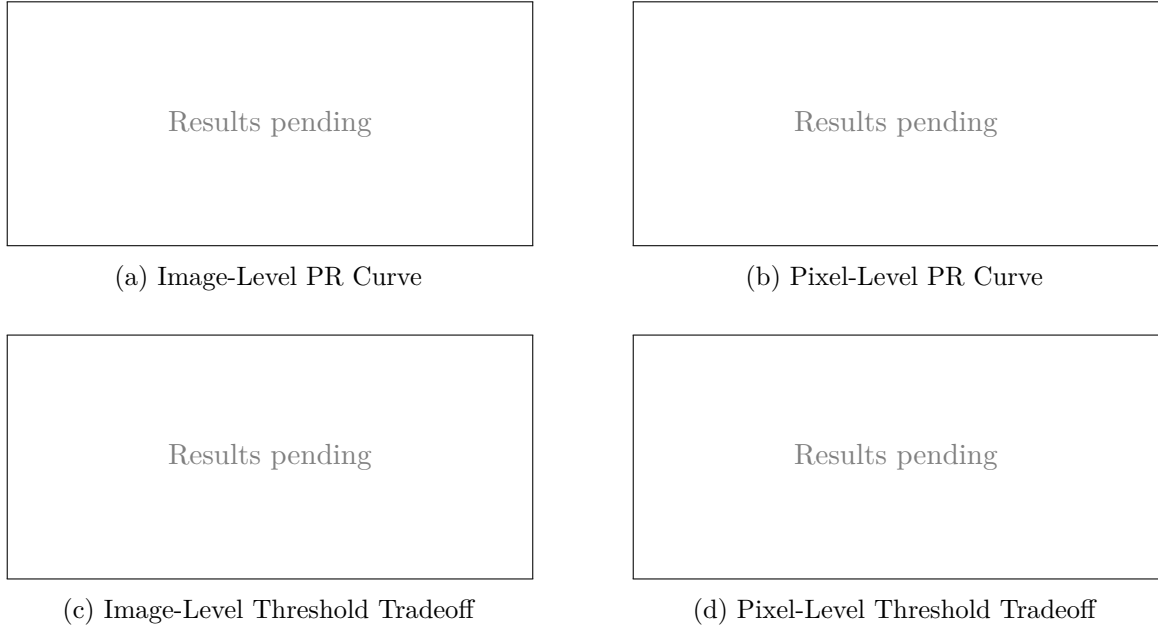


Figure 22: Evaluation curves for the *capsule* category using PatchCore (Custom/Tuned). The pixel-level AUPIMO is pending sweep completion.

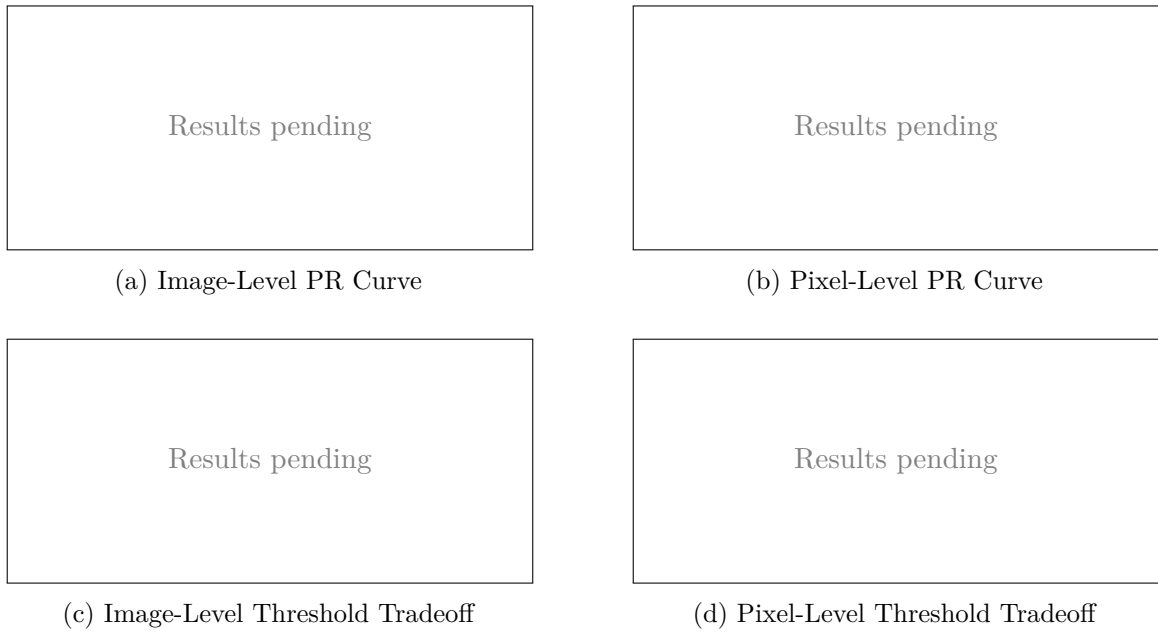
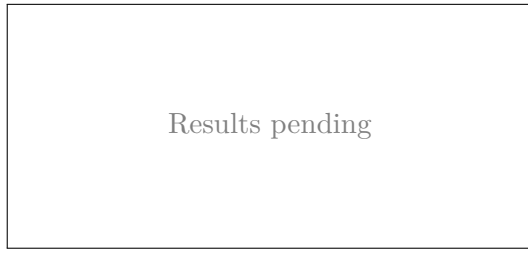
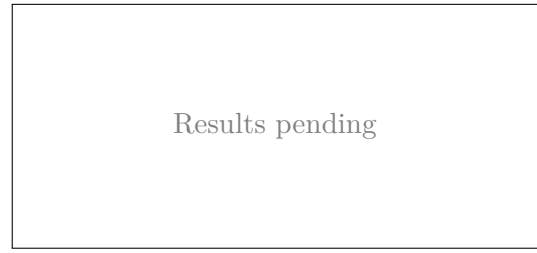


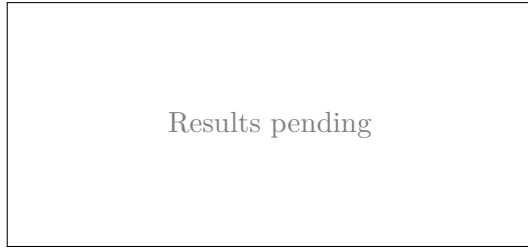
Figure 23: Evaluation curves for the *capsule* category using Keras CAE (Baseline). The pixel-level AUPIMO achieved is 0.037 (Best for this category: 0.037).



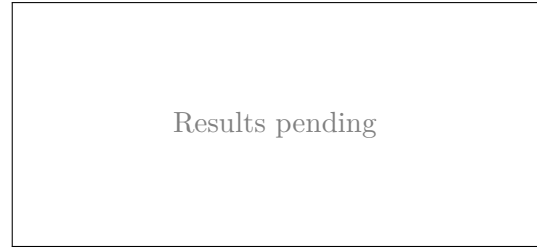
(a) Image-Level PR Curve



(b) Pixel-Level PR Curve



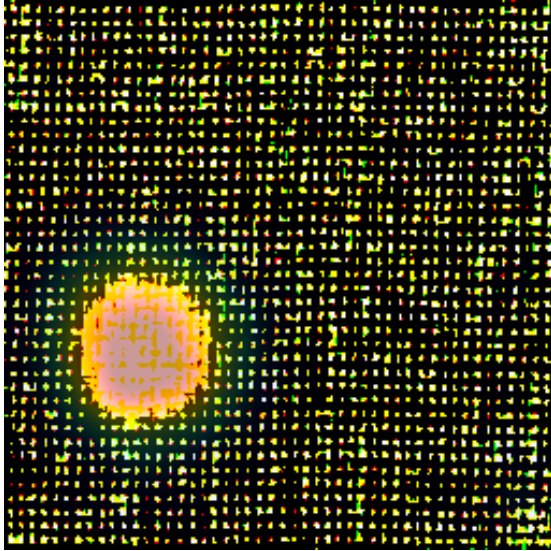
(c) Image-Level Threshold Tradeoff



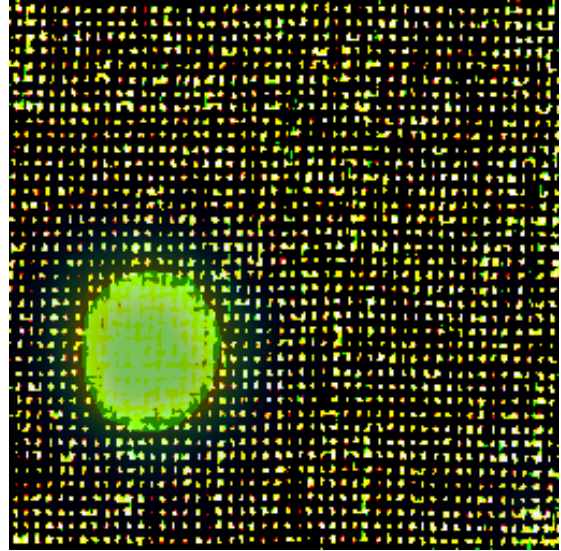
(d) Pixel-Level Threshold Tradeoff

Figure 24: Evaluation curves for the *capsule* category using Keras CAE (Tuned). The pixel-level AUPIMO achieved is 0.003 (Best for this category: 0.037).

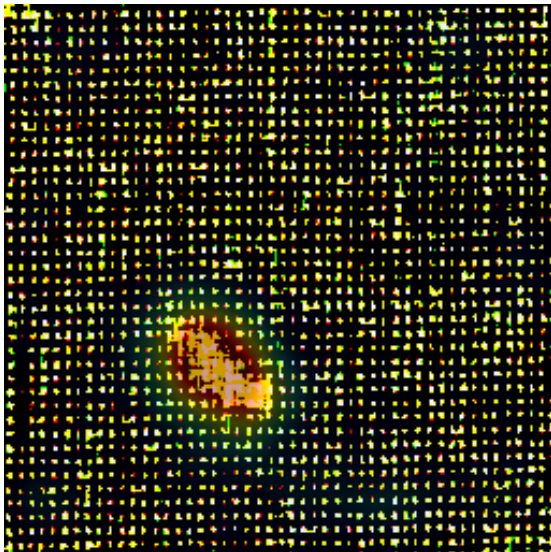
Visualisations and Curves: Carpet



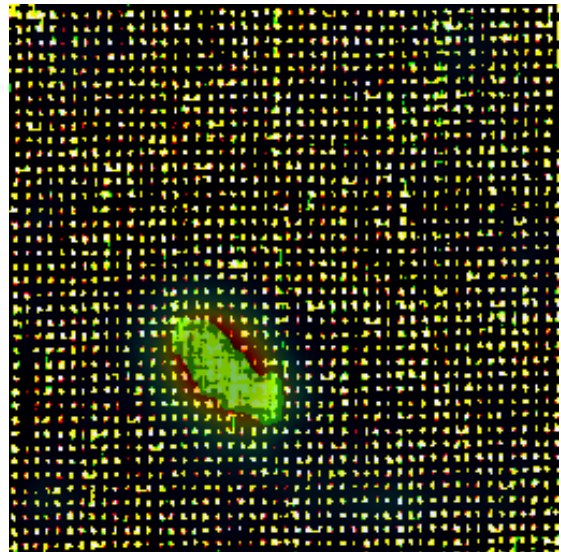
(a) Img 0: Pred



(b) Img 0: GT Overlay



(c) Img 1: Pred



(d) Img 1: GT Overlay

Figure 25: Qualitative results for Carpet using PatchCore (ResNet-18).

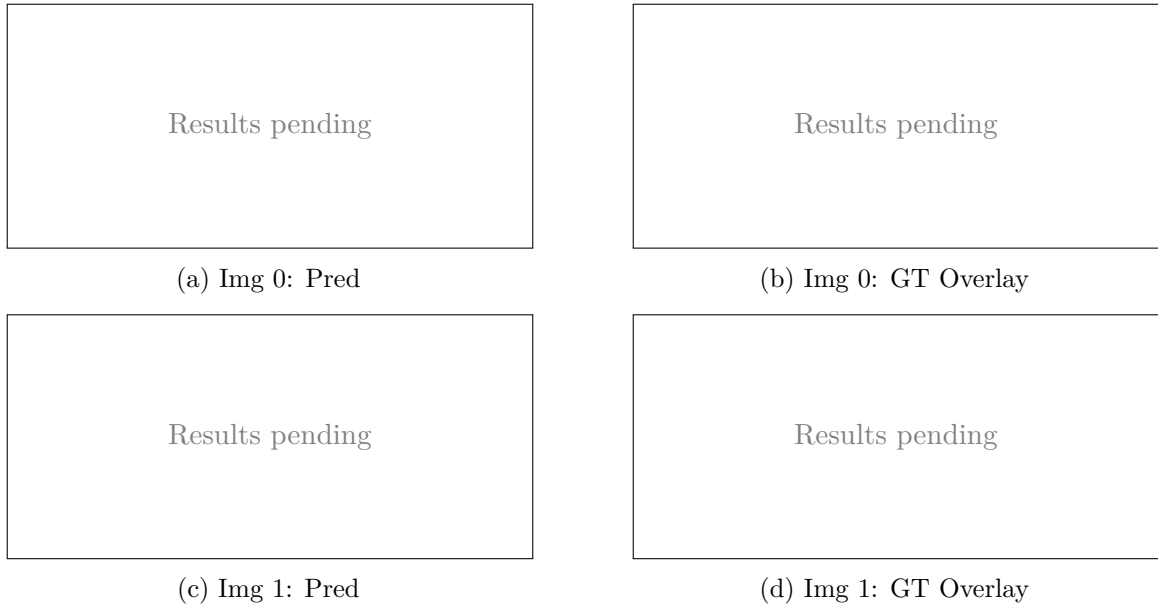


Figure 26: Qualitative results for Carpet using PatchCore (Custom/Tuned).

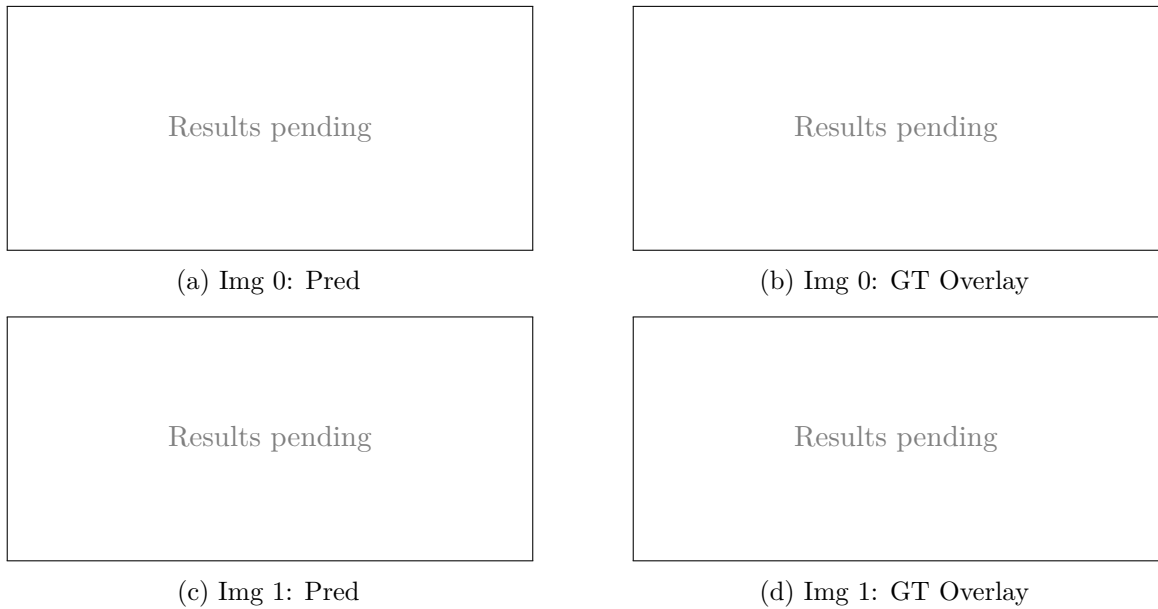


Figure 27: Qualitative results for Carpet using Keras CAE (Baseline).

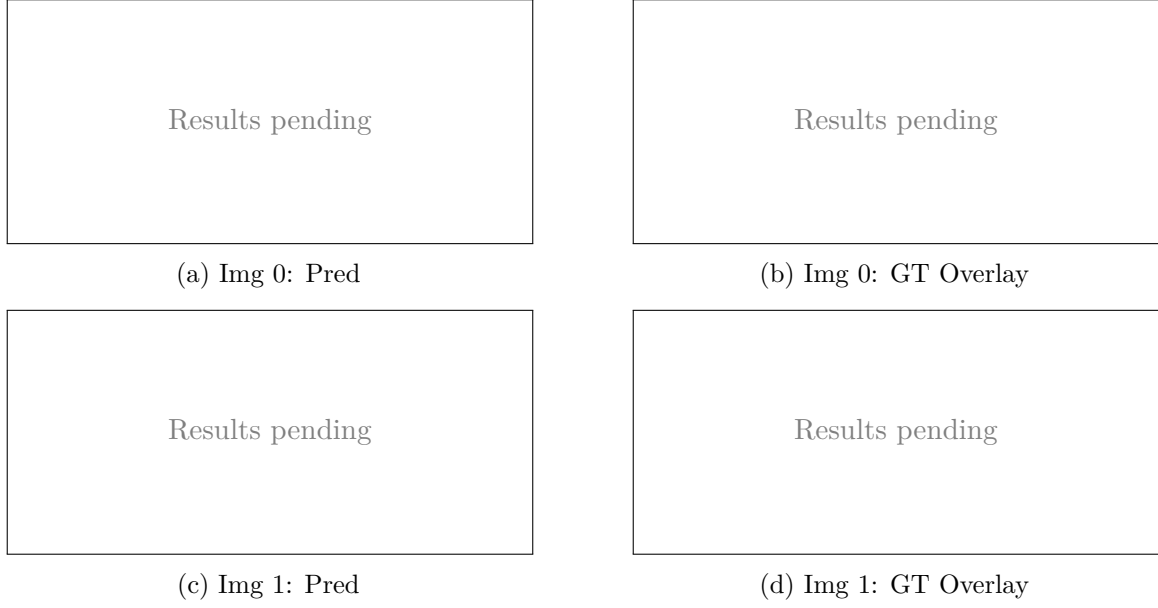


Figure 28: Qualitative results for Carpet using Keras CAE (Tuned).

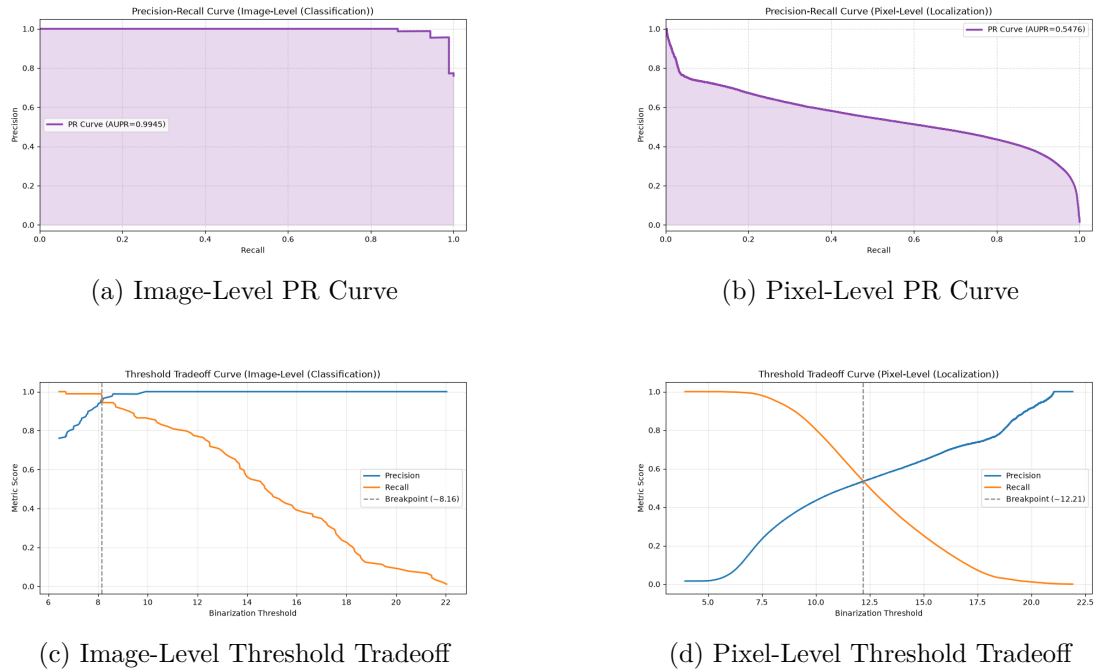


Figure 29: Evaluation curves for the *carpet* category using PatchCore (ResNet-18). The pixel-level AUPIMO achieved is 0.000 (Best for this category: 0.002).

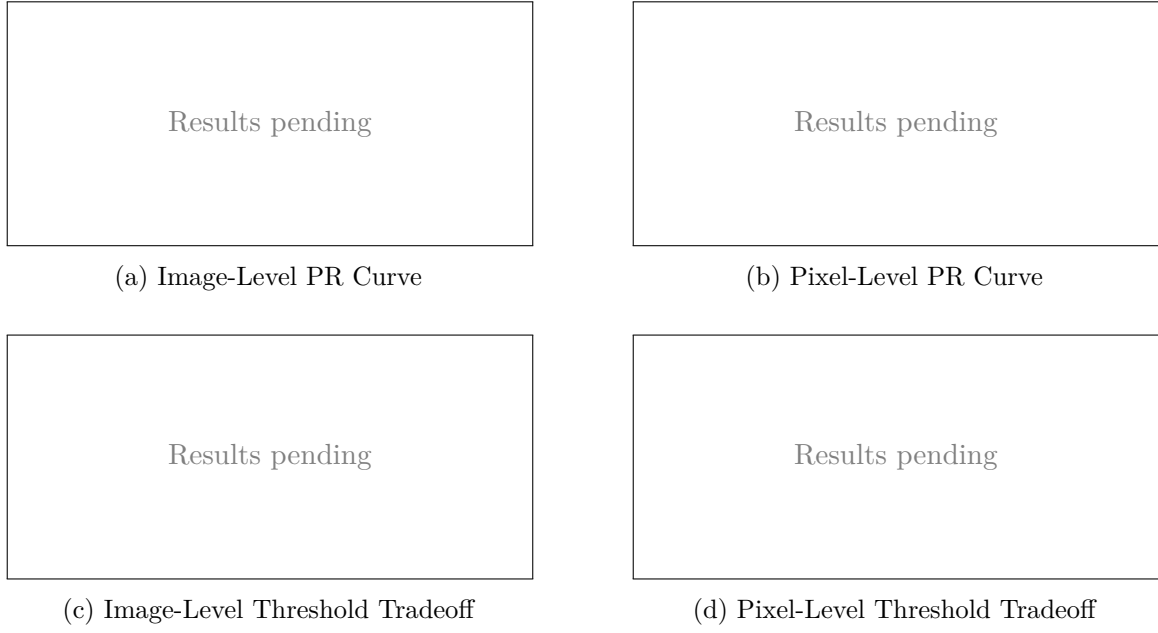


Figure 30: Evaluation curves for the *carpet* category using PatchCore (Custom/Tuned). The pixel-level AUPIMO is pending sweep completion.

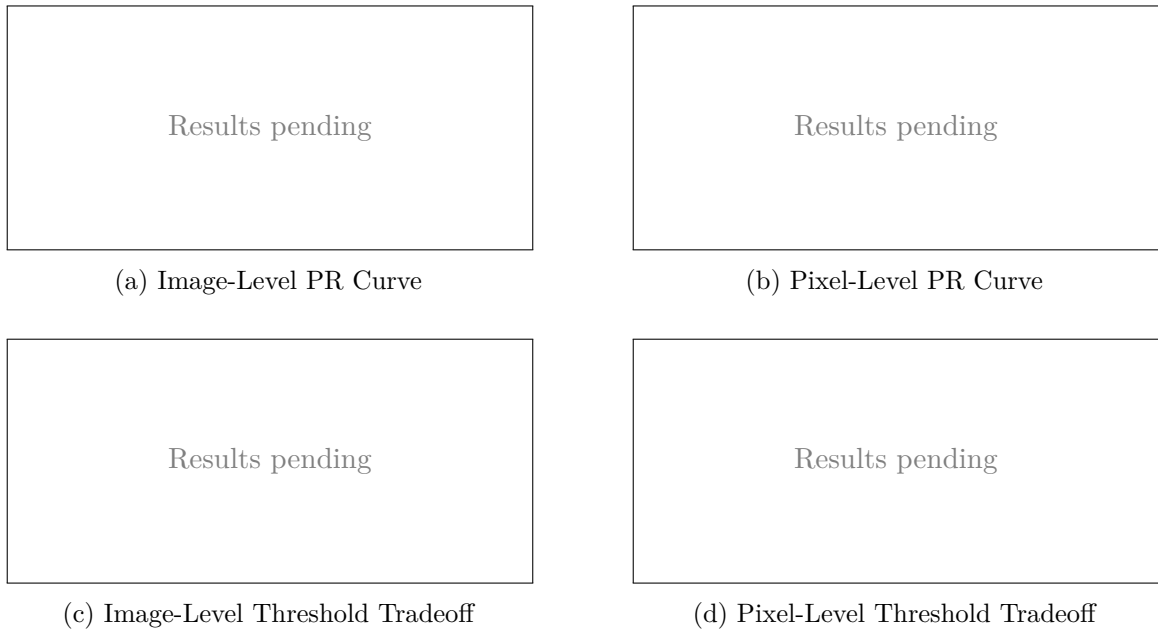
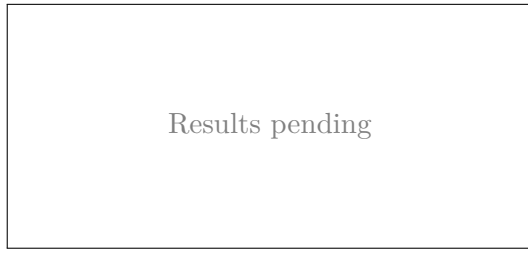
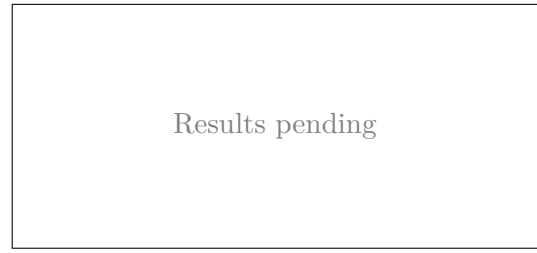


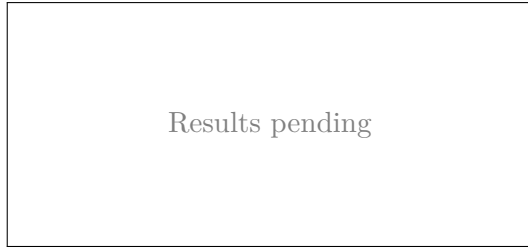
Figure 31: Evaluation curves for the *carpet* category using Keras CAE (Baseline). The pixel-level AUPIMO achieved is 0.002 (Best for this category: 0.002).



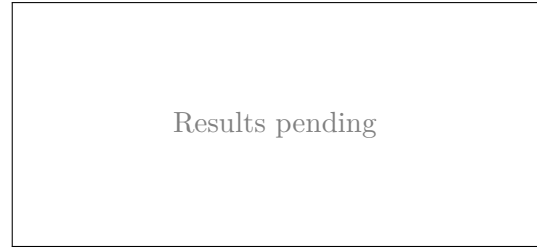
(a) Image-Level PR Curve



(b) Pixel-Level PR Curve



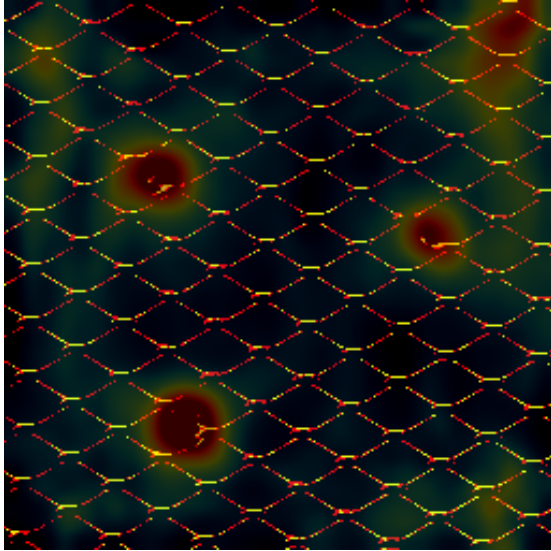
(c) Image-Level Threshold Tradeoff



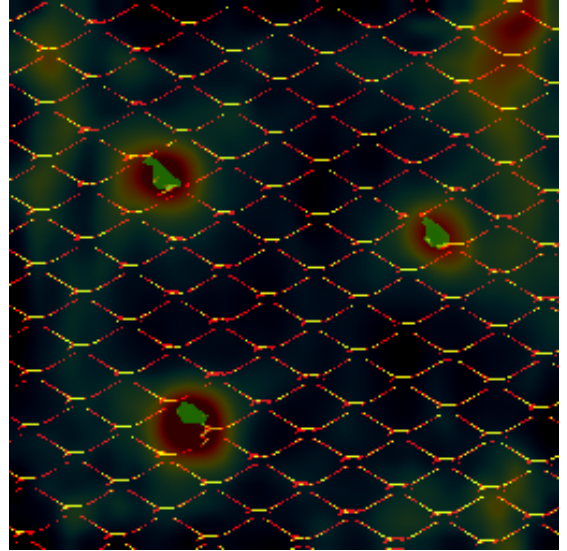
(d) Pixel-Level Threshold Tradeoff

Figure 32: Evaluation curves for the *carpet* category using Keras CAE (Tuned). The pixel-level AUPIMO achieved is 0.002 (Best for this category: 0.002).

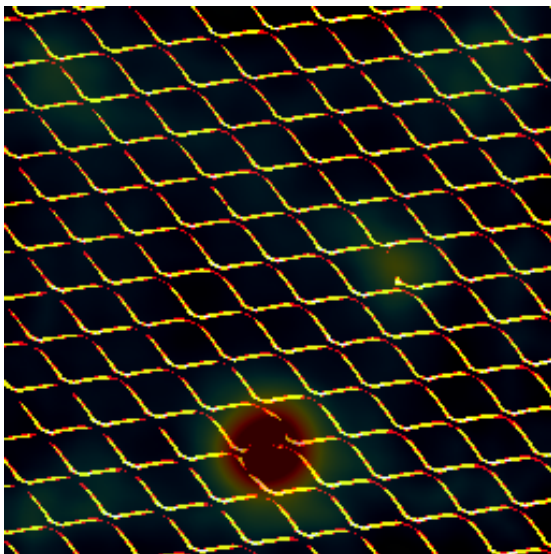
Visualisations and Curves: Grid



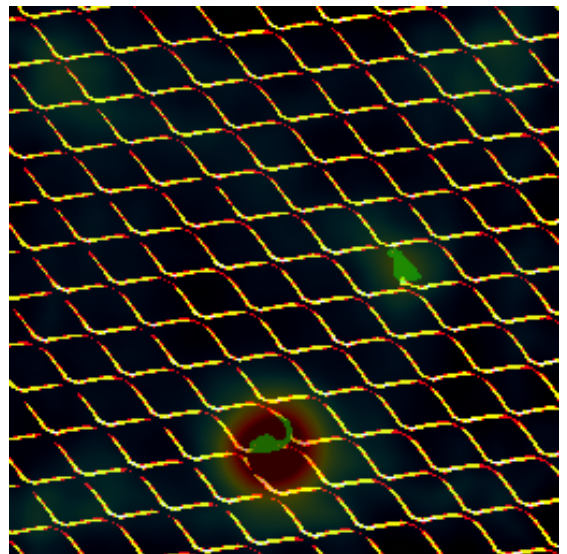
(a) Img 0: Pred



(b) Img 0: GT Overlay



(c) Img 1: Pred



(d) Img 1: GT Overlay

Figure 33: Qualitative results for Grid using PatchCore (ResNet-18).

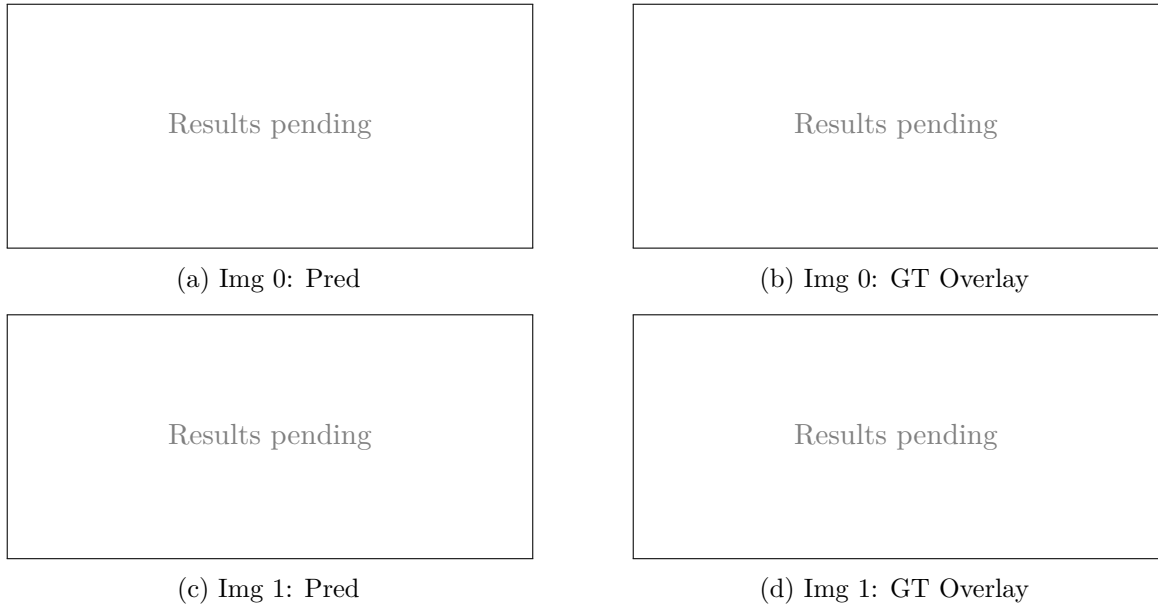


Figure 34: Qualitative results for Grid using PatchCore (Custom/Tuned).

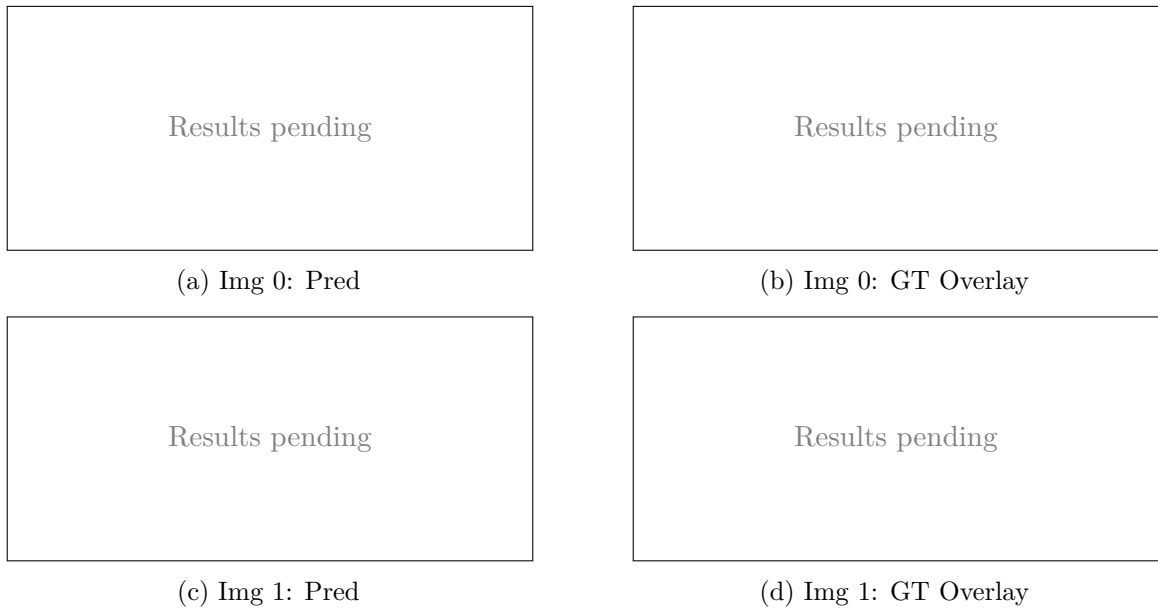


Figure 35: Qualitative results for Grid using Keras CAE (Baseline).

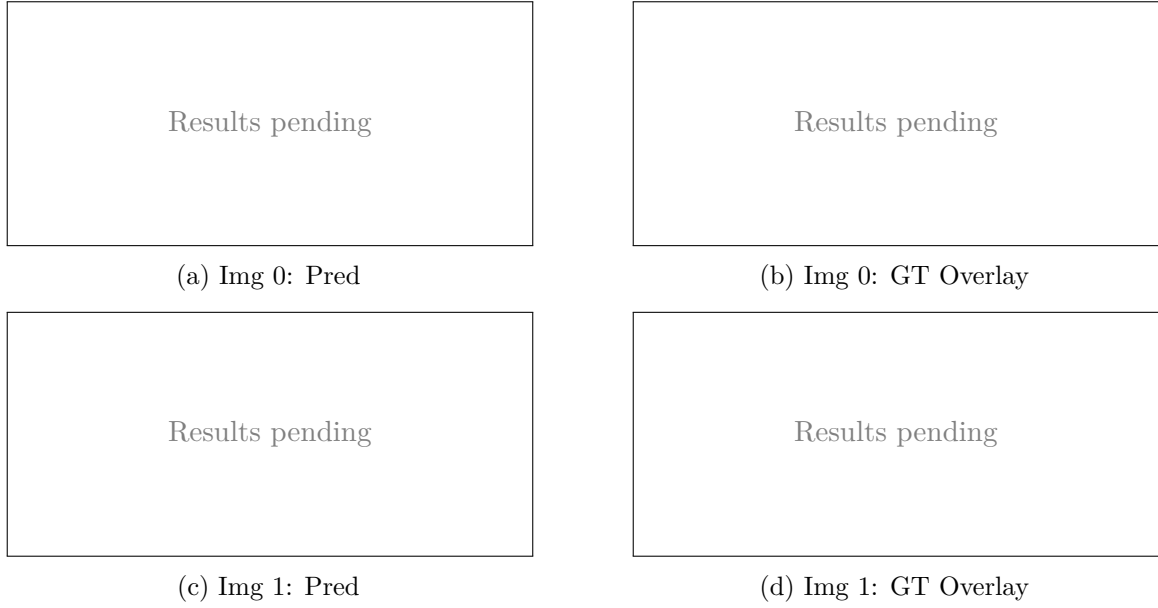
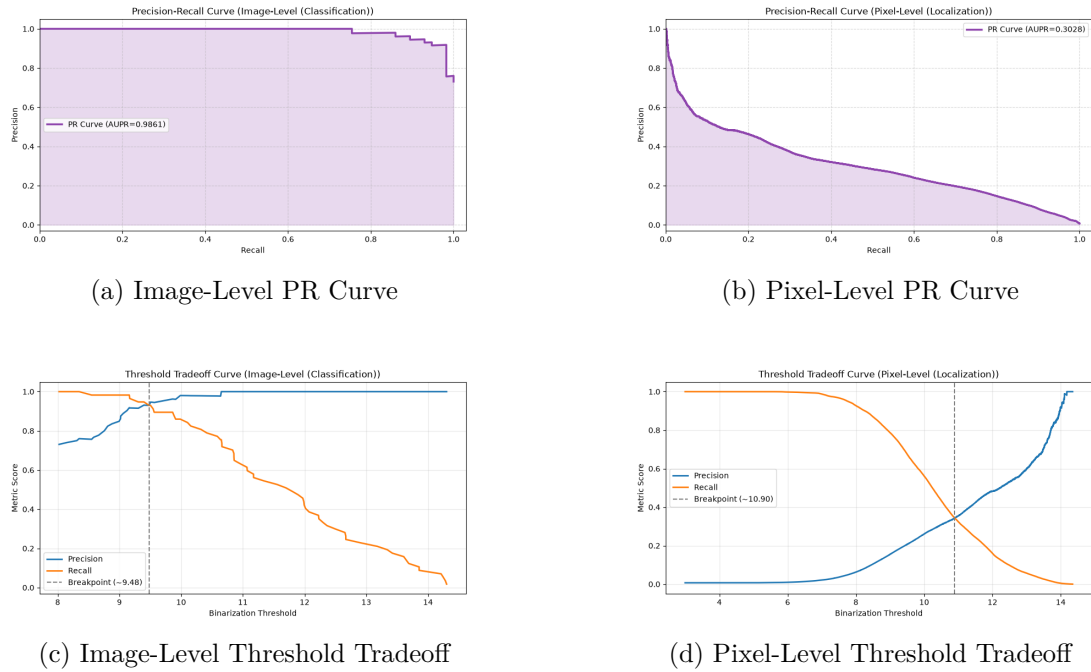


Figure 36: Qualitative results for Grid using Keras CAE (Tuned).


 Figure 37: Evaluation curves for the *grid* category using PatchCore (ResNet-18). The pixel-level AUPIMO achieved is 0.000 (Best for this category: 0.104).

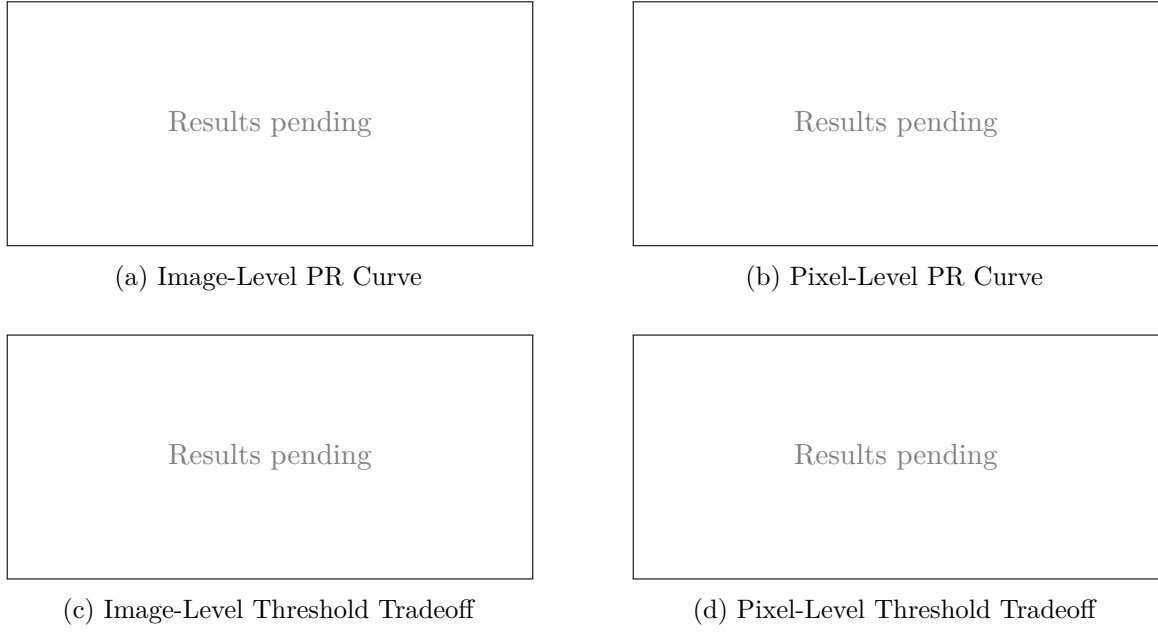


Figure 38: Evaluation curves for the *grid* category using PatchCore (Custom/Tuned). The pixel-level AUPIMO is pending sweep completion.

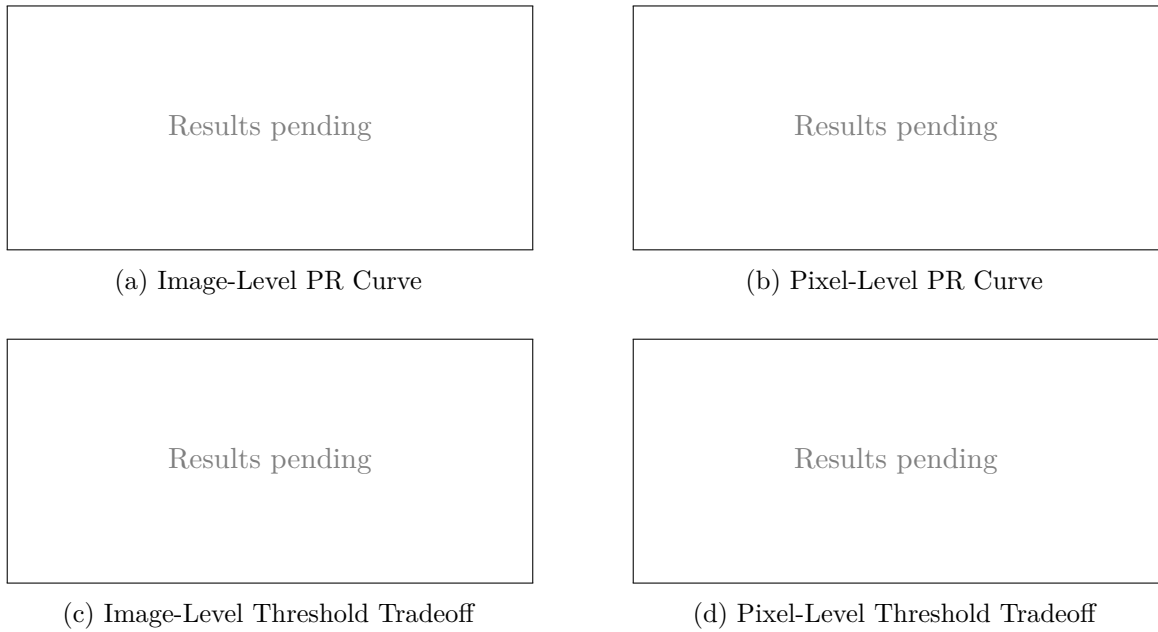


Figure 39: Evaluation curves for the *grid* category using Keras CAE (Baseline). The pixel-level AUPIMO achieved is 0.087 (Best for this category: 0.104).

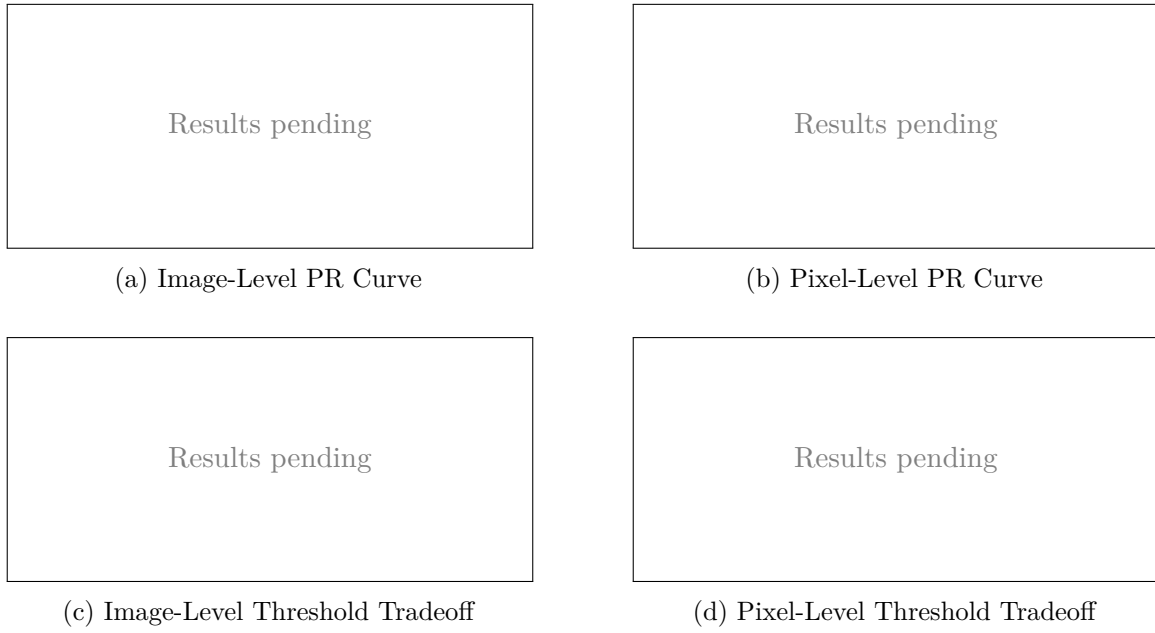
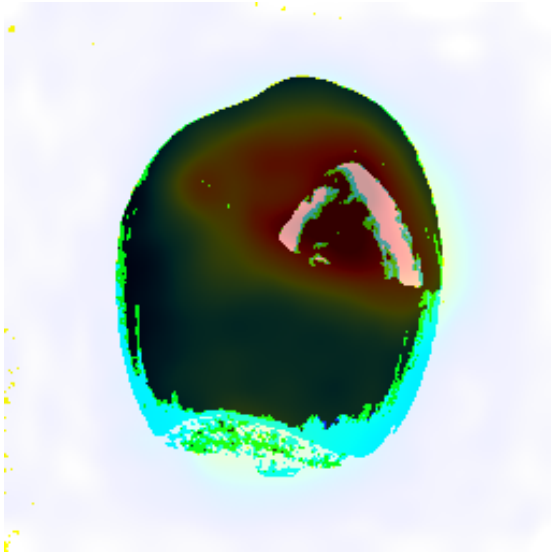
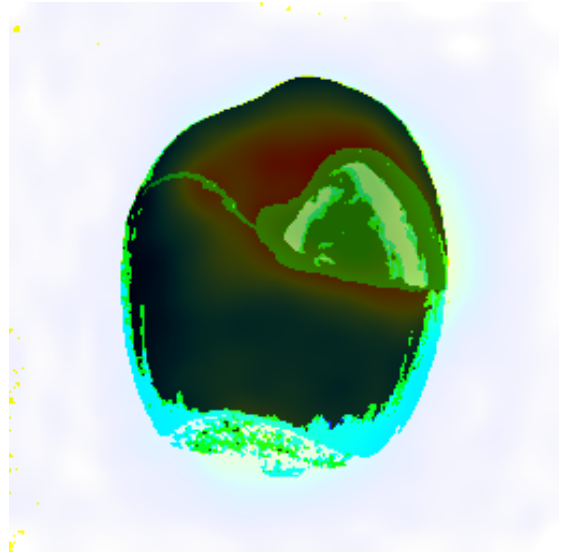


Figure 40: Evaluation curves for the *grid* category using Keras CAE (Tuned). The pixel-level AUPIMO achieved is 0.104 (Best for this category: 0.104).

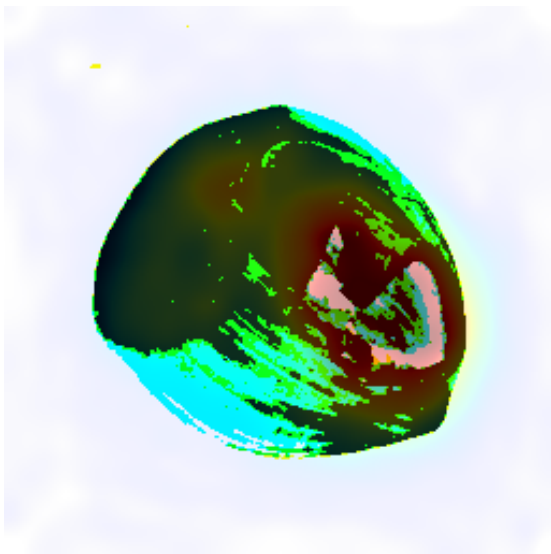
Visualisations and Curves: Hazelnut



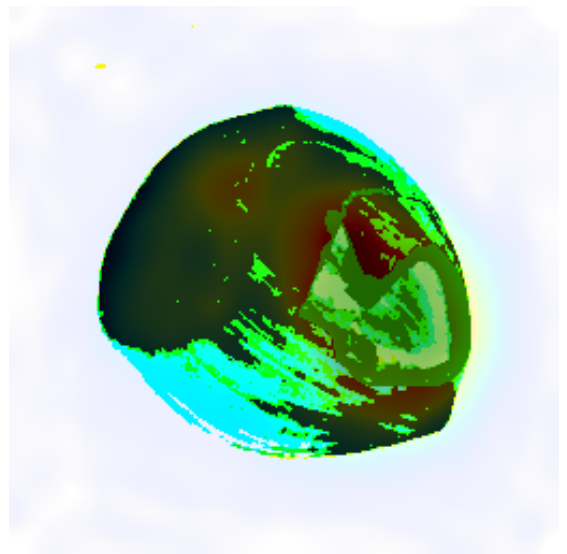
(a) Img 0: Pred



(b) Img 0: GT Overlay



(c) Img 1: Pred



(d) Img 1: GT Overlay

Figure 41: Qualitative results for Hazelnut using PatchCore (ResNet-18).

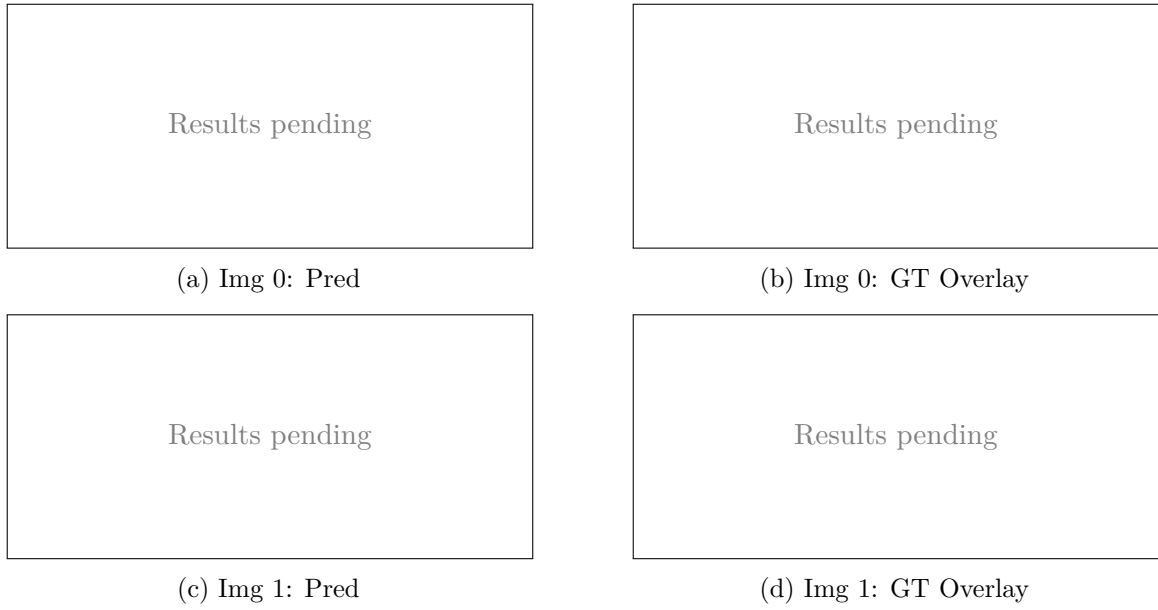


Figure 42: Qualitative results for Hazelnut using PatchCore (Custom/Tuned).

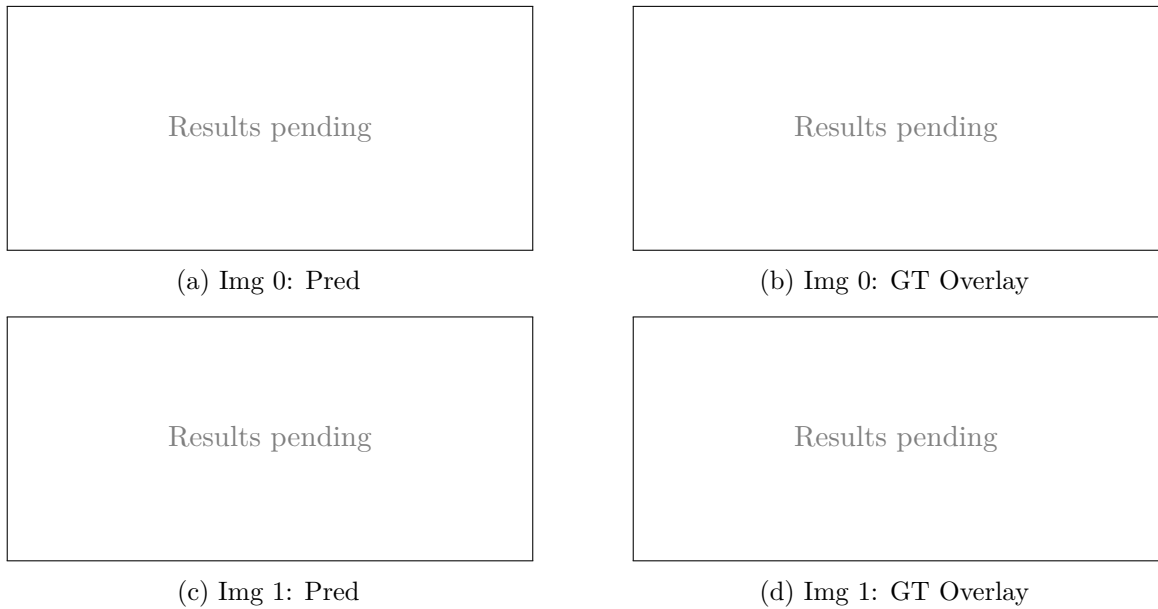


Figure 43: Qualitative results for Hazelnut using Keras CAE (Baseline).

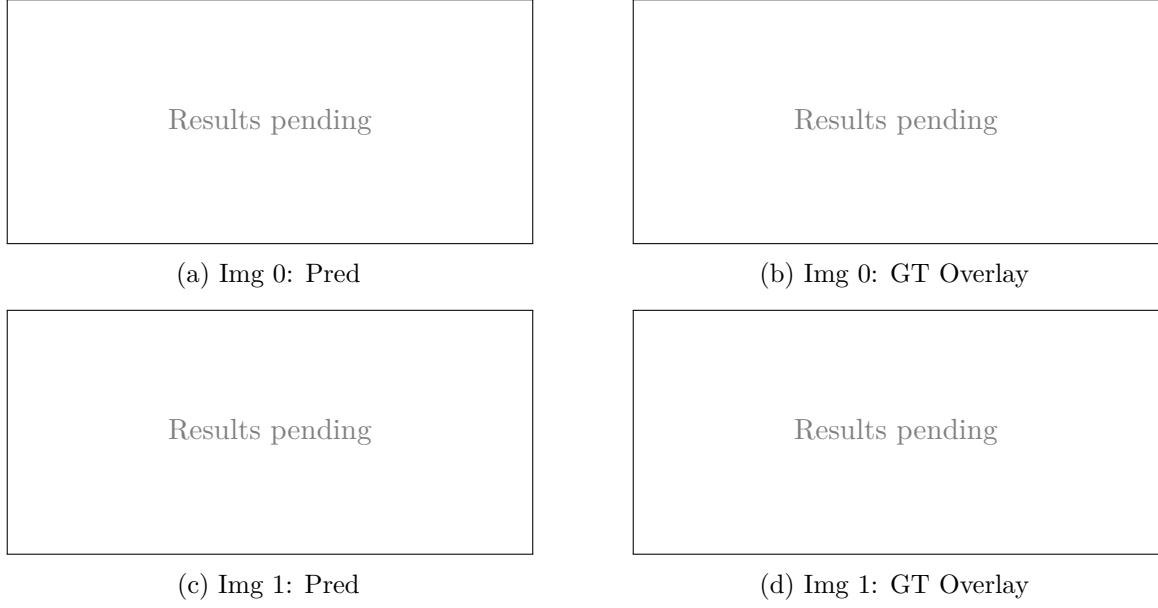


Figure 44: Qualitative results for Hazelnut using Keras CAE (Tuned).

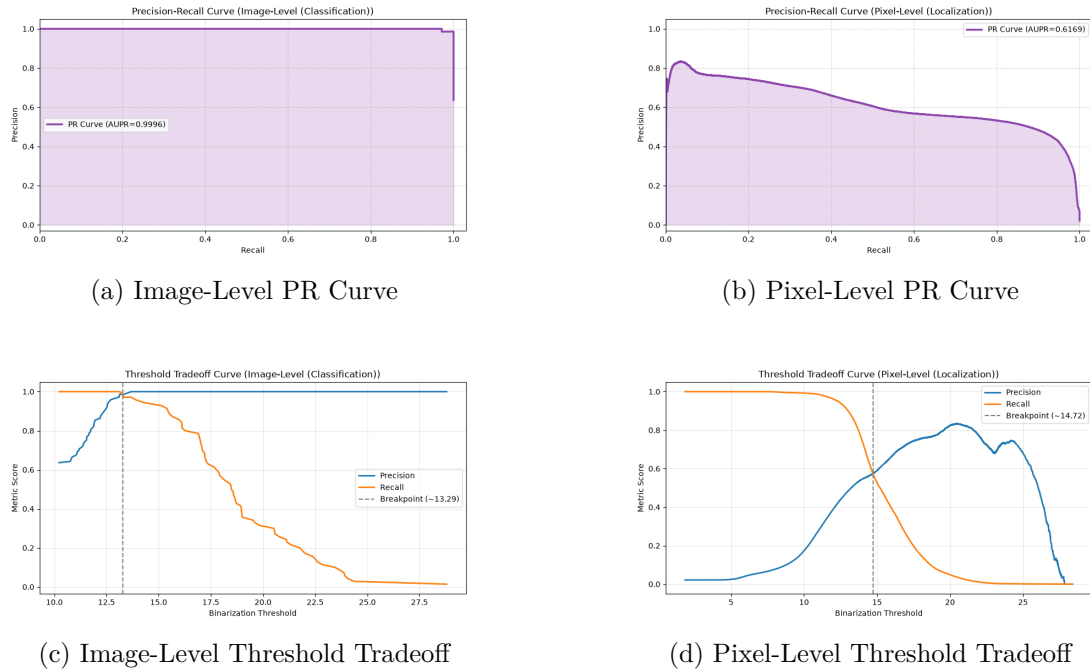


Figure 45: Evaluation curves for the *hazelnut* category using PatchCore (ResNet-18). The pixel-level AUPIMO achieved is 0.000 (Best for this category: 0.087).

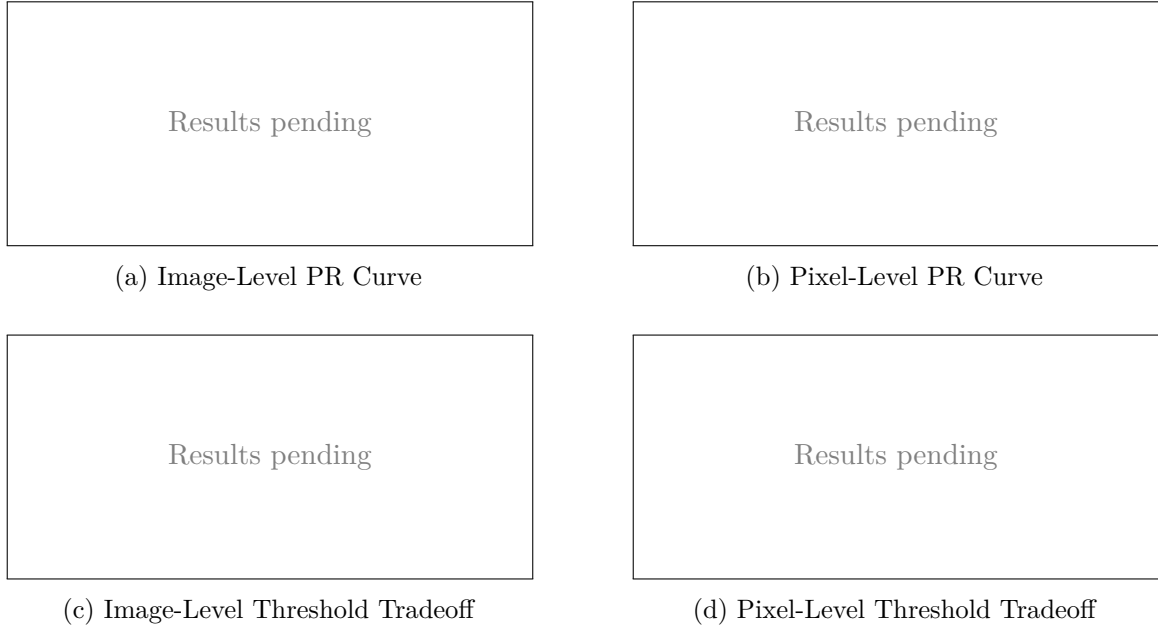


Figure 46: Evaluation curves for the *hazelnut* category using PatchCore (Custom/Tuned). The pixel-level AUPIMO is pending sweep completion.

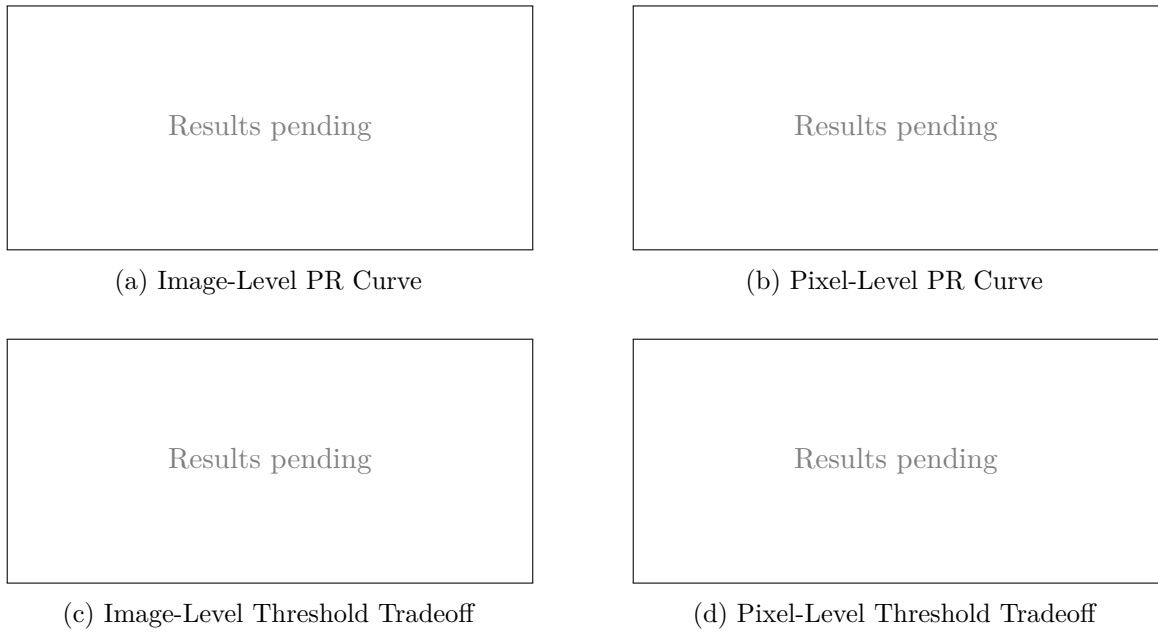


Figure 47: Evaluation curves for the *hazelnut* category using Keras CAE (Baseline). The pixel-level AUPIMO achieved is 0.087 (Best for this category: 0.087).

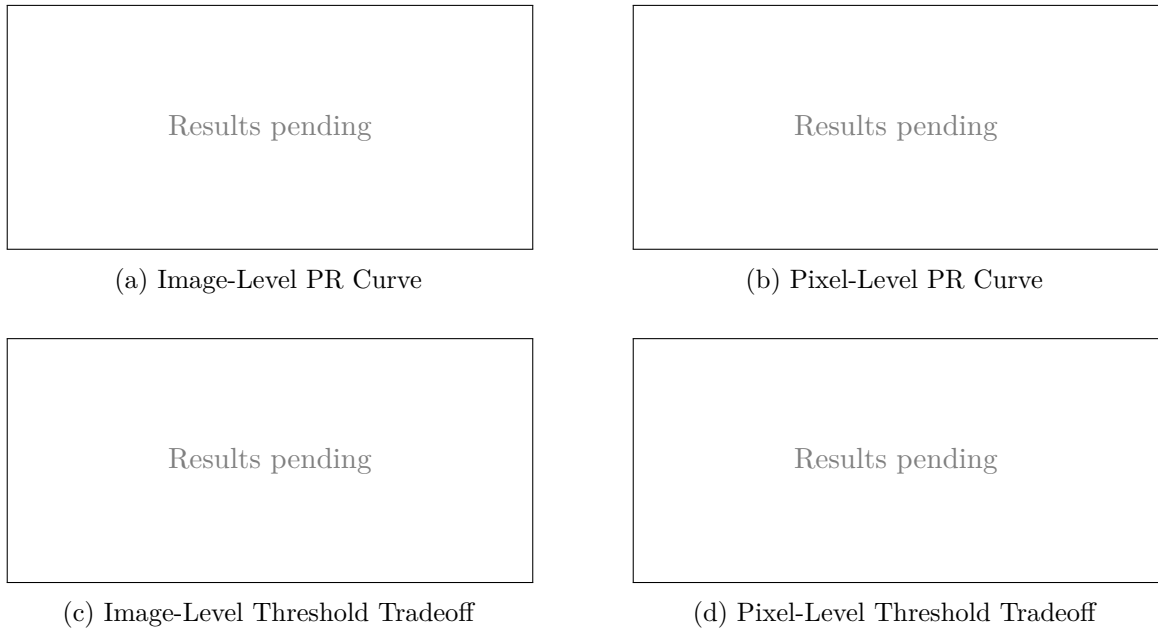


Figure 48: Evaluation curves for the *hazelnut* category using Keras CAE (Tuned). The pixel-level AUPIMO achieved is 0.054 (Best for this category: 0.087).